

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — Post-Launch and Live Operations Plan

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

DOCUMENTO

**25 /
25_Post_Launch_and_Live_Operations_Plan.md**

VERSIÓN

1.0

ESTADO

**PLANNING / GAME NOT RELEASED / LIVE
OPERATIONS NOT OPEN**

AUTORÍA

VRM Games / Blas Luis Rocha González

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 25_Post_Launch_and_Live_Operations_Plan.md
SHA-256 del Markdown: 158e1e43514c5d5d6c365f6cd1a73929f54c4a48c5cc460b593530ae5e49bac9
Tamaño del Markdown: 327,218 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — Post-Launch and Live Operations Plan
subtitle	Plan consolidado de soporte, incidentes, parches, comunidad, métricas, mantenimiento y fin de vida
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
document_version	1.0
project_version	0.0.17
status	PLANNING / GAME NOT RELEASED / LIVE OPERATIONS NOT OPEN

Índice

Cartridge & Cloud — Post-Launch and Live Operations Plan

Resumen ejecutivo

Convenciones de estado

0. Propósito, alcance y autoridad

1. Genealogía histórica

2. Diferencia entre soporte, live operations y desarrollo futuro

3. Estado actual del proyecto

4. Principios operativos

5. Horario y capacidad de soporte

6. Canales de entrada

7. Email de soporte

8. Steam Community Hub

9. Foro de bugs

10. Foro de sugerencias

11. Reseñas

12. Redes y web

13. Privacidad

14. Datos mínimos solicitados

15. Logs

16. Saves

17. Consentimiento

18. Clasificación de incidencias

19. Severidades S0–S4

20. Prioridades

21. Impacto

22. Frecuencia

23. Reproducibilidad

24. Alcance

25. Triage

26. Duplicados

27. Bugs conocidos

28. Workarounds

29. Escalado

30. Incidentes críticos

31. Caídas de lanzamiento

32. Pérdida de save

33. Corrupción

34. Crash

35. Bloqueo de progreso

36. Economía rota

37. Duplicación

38. Localización

39. Audio

40. Rendimiento

41. Compatibilidad

42. Instalación

43. Actualización

44. Antivirus

45. Steam Cloud

46. Proton y Steam Deck

47. Plantilla de reporte

48. Evidencia mínima

49. Player.log

50. Dumps

51. Hardware

52. Sistema operativo

- | | |
|---------------------------------|--|
| 53. Idioma | 85. Golden Path |
| 54. Versión | 86. Campaña de siete días |
| 55. Branch | 87. Save/load |
| 56. Manifest ID | 88. Instalación limpia |
| 57. Save afectado | 89. Actualización desde versión anterior |
| 58. Reproducción | 90. Rollback de versión |
| 59. Registro de incidencias | 91. Rollback de datos |
| 60. SLA internos | 92. SteamPipe |
| 61. Tiempo de primera respuesta | 93. Depots |
| 62. Tiempo de triage | 94. Manifest promotion |
| 63. Tiempo de decisión | 95. Notas de parche |
| 64. Comunicación inicial | 96. Idiomas de las notas |
| 65. Actualizaciones de estado | 97. Comunicación de riesgos |
| 66. Resolución | 98. Known issues |
| 67. Cierre | 99. Comunidad |
| 68. Hotfix | 100. Moderación |
| 69. Patch | 101. Código de conducta |
| 70. Update | 102. Spam |
| 71. Content update | 103. Abuso |
| 72. Save migration | 104. Contenido ilegal |
| 73. Backward compatibility | 105. Spoilers |
| 74. Forward compatibility | 106. Solicitudes de reembolso |
| 75. Versionado | 107. Comentarios negativos |
| 76. Branches | 108. Reseñas negativas |
| 77. Release candidate | 109. Reseñas positivas |
| 78. Previous stable | 110. Respuestas públicas |
| 79. Rollback | 111. Transparencia |
| 80. Kill switch o feature flag | 112. No prometer fechas sin aprobación |
| 81. Build reproducible | 113. Roadmap público |
| 82. Checksums | 114. Roadmap interno |
| 83. QA mínima de hotfix | 115. Sugerencias |
| 84. QA completa de patch | 116. Votaciones |

117. Alcance futuro	146. Informes semanales
118. DLC	147. Retrospectiva
119. Expansiones	148. Mantenimiento
120. Demo	149. Dependencias
121. Playtest	150. Unity y paquetes
122. Achievements	151. Vulnerabilidades
123. Cloud	152. Actualizaciones del motor
124. Localización futura	153. Compatibilidad futura
125. Nuevas plataformas	154. Pruebas de regresión
126. Métricas de lanzamiento	155. Deuda técnica
127. Crash-free sessions	156. End of life
128. Retención	157. Fin de soporte
129. Tiempo de juego	158. Aviso a jugadores
130. Abandono	159. Conservación de saves
131. Progreso	160. Disponibilidad offline
132. Economía	161. Archivado
133. Stockouts	162. Work packages
134. Softlocks	163. Gates
135. Rendimiento operativo	164. Checklists
136. Soporte	165. Plantillas
137. Volumen de tickets	166. Riesgos
138. Tiempo de respuesta	167. Glosario
139. Sentimiento	168. Historial de cambios
140. Reviews	Anexo A. Fuentes oficiales de Steamworks verificadas
141. Wishlists convertidas	Anexo B. Evidencia de implementación observada
142. Privacidad de métricas	Escenas de build observadas
143. Telemetría opcional	Archivos técnicos hashados
144. No recolectar datos sin política	Anexo C. Inventario de documentos consolidados
145. Informes diarios	Anexo D. Inventario histórico y operativo preservado
	Anexo E. Matriz de preparación postlanzamiento

Cartridge & Cloud — Post-Launch and Live Operations Plan

Proyecto: Cartridge & Cloud
Desarrollador: VRM Games / Blas Luis Rocha González
Plataforma prevista: PC / Steam
Motor: Unity 6.3 LTS 6000.3.18f1 / URP 17.3.0
Versión observada: 0.0.17
Estado: Sprints 0–15 CLOSED / PASS; Sprint 16 IN PROGRESS; Sprint 17 y H6 PENDING
Estado postlanzamiento: NOT OPEN
Fecha de verificación de Steamworks: 1 de julio de 2026

Aviso: este documento es un plan operativo y de gobernanza. No sustituye asesoramiento jurídico, laboral, fiscal, de protección de datos, seguridad o comunicación de crisis. Los términos y herramientas de Steamworks deben revalidarse antes de cada activación material.

Resumen ejecutivo

El proyecto todavía no está publicado y no dispone de AppID, Steamworks SDK, Community Hub, Steam Cloud, telemetría, crash reporting externo, ticketing ni dirección pública de soporte. Por tanto, este plan no describe una operación existente: define las condiciones, procedimientos, plantillas y gates que deberán estar preparados antes de aceptar jugadores públicos.

La base técnica aporta ventajas importantes: guardado local atómico, backup y recuperación; estado económico determinista; baseline de QA; versionado; builds históricas; y un plan de Steam detallado. Sin embargo, la publicación sigue bloqueada por StoreInitial, Sprint 17/H6, localización, audio, fuente, notices, clearance, soporte y autorización legal.

Área	Estado actual	Decisión
Juego público	No publicado	NOT OPEN
Community Hub	No existe	Preparar antes de Coming Soon
Soporte público	No existe	Definir email/Hub/ticketing mínimo
Telemetría	No integrada	NOT OPEN hasta plan de privacidad
Crash reporting	No integrado	Evaluar solución moderna; Steam legacy no apto para x64

Área	Estado actual	Decisión
Steam Cloud	No integrado	Evaluar después de migración y conflicto de saves
Save local	Atómico, backup, recovery, schema 2	Base válida, migradores pendientes
Builds	17 PASS + 3 pendientes	Ninguna pública/elegible
Rollback Steam	Documentado, no probado	Requiere AppID/branches/manifests
Baseline de regresión	1215 EditMode + 70 PlayMode	Debe ampliarse con campañas postlanzamiento

Convenciones de estado

Estado	Significado
CURRENT	Evidencia real observada en proyecto o documento vigente.
PLANNED	Trabajo aprobado como plan, aún no ejecutado.
DEFERRED	Válido, pero pospuesto por prioridad o dependencia.
NOT OPEN	Sistema o compromiso no autorizado.
BLOCKED	No puede avanzar hasta cerrar una condición.
HISTORICAL	Se conserva como genealogía, no como autoridad actual.

0. Propósito, alcance y autoridad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **propósito, alcance y autoridad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: [ST-OPS-001](#); [ST-OPS-002](#); [ST-OPS-014](#).

1. Genealogía histórica

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **genealogía histórica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: [ST-OPS-001](#); [ST-OPS-002](#); [ST-OPS-014](#).

2. Diferencia entre soporte, live operations y desarrollo futuro

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **diferencia entre soporte, live operations y desarrollo futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Diferencia entre soporte, live operations y desarrollo futuro** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: **ST-OPS-001**; **ST-OPS-002**; **ST-OPS-014**.

3. Estado actual del proyecto

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **estado actual del proyecto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La aplicación observada es 0.0.17, Windows x64, Unity 6000.3.18f1. Sprints 0-15 y BLD-016-PRE están en PASS; Sprint 16 post-integración, Sprint 17 y H6 están pendientes. El registro legal cuenta 0 builds elegibles para publicación, 9 riesgos High/Critical y 7 deudas S1.

Regla operativa. No se denomina postlanzamiento a una fase que todavía no tiene producto publicado, AppID, branch pública ni autorización legal.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de Estado actual del proyecto y su relación con versión, build, save y comunicación.

Evidencia mínima. Estado consolidado de producción, legal, Steam y QA con fecha y hash.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

4. Principios operativos

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **principios operativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.

2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Principios operativos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: **ST-OPS-001; ST-OPS-002; ST-OPS-014.**

5. Horario y capacidad de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **horario y capacidad de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Horario y capacidad de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

6. Canales de entrada

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **canales de entrada** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Canales de entrada** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

7. Email de soporte

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **email de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe una dirección pública documentada. Debe crearse una cuenta del dominio oficial solo después de cerrar identidad, dominio, privacidad y capacidad de atención.

Regla operativa. No se publica una dirección personal ni se usa como repositorio de saves indefinido. Se configura MFA, recuperación y retención.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Email de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba de envío/recepción, autorespuesta, acceso, backup y política de datos.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

8. Steam Community Hub

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **steam community hub** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El Hub aparecerá cuando la aplicación alcance Coming Soon. Steam permite foros específicos para bugs o idiomas y muestra como desarrollador a las cuentas con permisos adecuados.

Regla operativa. Antes de Coming Soon deben existir reglas, permisos, owner y un plan de moderación. Valve revisa contenido reportado, pero el estudio mantiene responsabilidad operativa.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.

5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de [Steam Community Hub](#) y su relación con versión, build, save y comunicación.

Evidencia mínima. Captura de Hub, foros, permisos, reglas y prueba de moderación.

Fuentes Steamworks relacionadas: [ST-OPS-001](#); [ST-OPS-002](#); [ST-OPS-014](#).

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

9. Foro de bugs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de bugs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de [Foro de bugs](#) y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: [ST-OPS-001](#); [ST-OPS-002](#); [ST-OPS-014](#).

10. Foro de sugerencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **foro de sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Foro de sugerencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: **ST-0PS-001**; **ST-0PS-002**; **ST-0PS-014**.

11. Reseñas

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **reseñas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Las reviews son feedback sobre juego, valor, prácticas y comunidad. No todas requieren respuesta; una respuesta oficial puede amplificar una discusión.

Regla operativa. No solicitar reviews dentro del juego ni ofrecer recompensas. No refutar opiniones. Responder solo con claridad, corrección factual, solución o enlace a notas.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de [Reseñas](#) y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de review, decisión de responder, wording aprobado y resultado.

Fuentes Steamworks relacionadas: [ST-OPS-001](#); [ST-OPS-002](#); [ST-OPS-014](#).

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

12. Redes y web

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **redes y web** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.

4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Redes y web** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

13. Privacidad

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **privacidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No hay telemetría, Cloud ni servicio online integrado; por ello la superficie de datos actual es principalmente local. El soporte futuro puede recibir logs y saves si el jugador los envía voluntariamente.

Regla operativa. Aplicar minimización, propósito, retención y acceso restringido. No prometer anonimato si los archivos contienen identificadores o rutas.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Privacidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Inventario de datos, notice de soporte, retención y procedimiento de borrado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

14. Datos mínimos solicitados

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **datos mínimos solicitados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Datos mínimos solicitados** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: **ST-OPS-001**; **ST-OPS-002**; **ST-OPS-014**.

15. Logs

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **logs** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El artefacto primario es **Player.log**; el proyecto también conserva logs y reportes de builds internas. No hay envío automático.

Regla operativa. El log se pide con instrucciones por plataforma y se revisa para retirar nombres de usuario, rutas o información ajena al incidente.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Logs** y su relación con versión, build, save y comunicación.

Evidencia mínima. Log hashado, versión, ventana temporal y análisis asociado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-014.

16. Saves

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los guardados integrados viven bajo `Application.persistentDataPath/IntegratedSaveGames`, usan primary, `.bak`, `.tmp` y `.recovery`, generación creciente y validación. El esquema actual es `2`.

Regla operativa. Un save enviado no se modifica sobre la única copia. Se duplica, se trabaja en entorno aislado y se conserva cadena de custodia.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Saves** y su relación con versión, build, save y comunicación.

Evidencia mínima. Save original hashado, copia de trabajo, resultado de validación y herramienta usada.

Fuentes Steamworks relacionadas: ST-0PS-001; ST-0PS-002; ST-0PS-014.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

17. Consentimiento

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **consentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Consentimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: ST-0PS-001; ST-0PS-002; ST-0PS-014.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

18. Clasificación de incidencias

Este capítulo forma parte del bloque **gobernanza y canales** y define cómo se gobierna **clasificación de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El juego no está publicado y no existe soporte público operativo. No hay Community Hub, email de soporte, ticketing, telemetría, crash SDK ni Steamworks en el código o paquetes. La operación se diseña ahora para que pueda activarse de forma controlada después de una candidata pública.

Regla operativa. Ningún canal se anuncia hasta que exista owner, horario, plantilla, política de privacidad y capacidad real de responder. Los canales oficiales deben ser mínimos, reconocibles y trazables; nunca se improvisan desde cuentas personales sin control de acceso.

Procedimiento mínimo.

1. Definir owner principal y suplente.
2. Registrar horario realista en Europe/Madrid y expectativas, no promesas de atención 24/7.
3. Crear identidad y permisos separados para soporte/moderación.
4. Publicar qué información puede enviar el jugador y qué debe ocultar.
5. Ensayar el flujo completo con un ticket ficticio antes de abrirlo.
6. Registrar en el ticket o release record el impacto específico de **Clasificación de incidencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de canales, permisos, horarios, plantillas, política de datos y prueba de extremo a extremo.

Fuentes Steamworks relacionadas: **ST-OPS-001**; **ST-OPS-002**; **ST-OPS-014**.

19. Severidades S0-S4

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **severidades s0-s4** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Severidades S0-S4** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

Severidad	Ejemplo postlanzamiento	Respuesta
S0	riesgo de seguridad, pérdida masiva de datos o build dañina	retirada/rollback inmediato y comunicación de crisis
S1	no arranca, corrupción reproducible, softlock general, exploit económico persistente	congelar release, hotfix/rollback prioritario
S2	feature importante rota con workaround limitado	patch prioritario y known issue
S3	defecto menor o localizado	backlog de patch
S4	cosmético, sugerencia o mejora	priorización ordinaria

20. Prioridades

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **prioridades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Prioridades** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

21. Impacto

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **impacto** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.

4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Impacto** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006 .

22. Frecuencia

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **frecuencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Frecuencia** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006 .

23. Reproducibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **reproducibilidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Reproducibilidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006.

24. Alcance

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **alcance** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Alcance** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

25. Triage

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **trriage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.

5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Triage** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

26. Duplicados

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicados** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Duplicados** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

27. Bugs conocidos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bugs conocidos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Bugs conocidos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006.

28. Workarounds

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **workarounds** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Workarounds** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

29. Escalado

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **escalado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.

4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Escalado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

30. Incidentes críticos

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **incidentes críticos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Incidentes críticos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

31. Caídas de lanzamiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **caídas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Caídas de lanzamiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**.

32. Pérdida de save

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **pérdida de save** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Es un incidente potencial S0/S1 porque afecta progreso y confianza. El repositorio puede recuperar backup, pero una pérdida más allá de primary/backup necesita procedimiento manual y comunicación prioritaria.

Regla operativa. Congelar acciones que sobrescriban archivos, pedir copia completa de la carpeta y evitar instrucciones destructivas.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Pérdida de save** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete primary/backup/recovery, logs, versión, causa y plan de restauración.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

33. Corrupción

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **corrupción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El codec rechaza schema no soportado, generación inválida y datos inconsistentes; el repositorio intenta backup antes de declarar **CorruptPrimaryNoBackup** .

Regla operativa. No “arreglar” JSON a mano sin herramienta versionada, validación y backup. Toda reparación debe producir un nuevo archivo y registro.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.

6. Registrar en el ticket o release record el impacto específico de **Corrupción** y su relación con versión, build, save y comunicación.

Evidencia mínima. Diagnóstico de codec, herramienta de reparación, tests y save restaurado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

34. Crash

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **crash** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe SDK externo. La función histórica Steam Error Reporting no es apropiada para el objetivo Windows x64 porque la documentación oficial la marca cercana a EOL y limitada a 32 bits.

Regla operativa. Elegir explícitamente entre soporte manual, dumps del sistema o un proveedor moderno, con revisión legal y de privacidad.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Crash** y su relación con versión, build, save y comunicación.

Evidencia mínima. Crash reproducible, log/dump, símbolos, build exacta, stack y fix validado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

35. Bloqueo de progreso

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **bloqueo de progreso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Bloqueo de progreso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

36. Economía rota

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **economía rota** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Economía rota** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**.

37. Duplicación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **duplicación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Duplicación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

38. Localización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **localización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Localización** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

39. Audio

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **audio** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Audio** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-005; ST-0PS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

40. Rendimiento

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **rendimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Rendimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

41. Compatibilidad

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **compatibilidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Compatibilidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

42. Instalación

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **instalación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Instalación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006 .

43. Actualización

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **actualización** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Actualización** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

44. Antivirus

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **antivirus** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Antivirus** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

45. Steam Cloud

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **steam cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No está implementado. El guardado actual es local y atómico. Steam Cloud puede usar Auto-Cloud o API, sincroniza antes/después de sesión y puede probarse en modo developers-only.

Regla operativa. No activar Cloud hasta probar conflictos, backup, cuota, múltiples equipos, desactivación por usuario, suspend/resume y compatibilidad de schema.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Steam Cloud** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de paths, cuotas, pruebas en dos equipos, **cloud_log.txt** y decisión.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006.

46. Proton y Steam Deck

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **proton y steam deck** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsibles son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.
2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de **Proton y Steam Deck** y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: **ST-OPS-004**; **ST-OPS-005**; **ST-OPS-006**.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

47. Plantilla de reporte

Este capítulo forma parte del bloque **clasificación e incidentes** y define cómo se gobierna **plantilla de reporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El proyecto ya usa severidades y gates en QA, pero no existe una mesa postlanzamiento activa. Los mayores riesgos previsible son pérdida/corrupción de save, bloqueo de progreso, duplicación económica, crash, instalación/actualización y regresiones de StoreInitial.

Regla operativa. La severidad se asigna por impacto observable, alcance y recuperabilidad, no por volumen emocional. Pérdida de save, corrupción generalizada, imposibilidad de iniciar y exploits económicos persistentes son candidatos S0/S1 y pueden exigir retirada o rollback.

Procedimiento mínimo.

1. Capturar versión, branch, manifest, plataforma, idioma y ruta de reproducción.

2. Separar incidente, defecto, consulta, sugerencia y abuso.
3. Buscar duplicados antes de abrir trabajo nuevo.
4. Definir workaround seguro sin alterar datos críticos.
5. Escalar S0/S1 al owner de release y congelar publicaciones no relacionadas.
6. Registrar en el ticket o release record el impacto específico de `Plantilla de reporte` y su relación con versión, build, save y comunicación.

Evidencia mínima. Ticket reproducible, evidencia, clasificación, owner, decisión y vínculo a build/save afectados.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`.

```
Título:
Versión / build / branch / manifest:
Sistema operativo y hardware:
Idioma y resolución:
Pasos exactos:
Resultado esperado:
Resultado real:
Frecuencia:
Player.log adjunto: sí/no
Save adjunto y consentimiento: sí/no
Workaround probado:
Notas:
```

48. Evidencia mínima

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **evidencia mínima** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de **Evidencia mínima** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: **ST-OPS-012**; **ST-OPS-015**.

49. Player.log

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **player.log** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de **Player.log**, builds versionadas internamente y guardados JSON con primary, **.bak**, **.tmp** y **.recovery**. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de **Player.log** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

50. Dumps

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **dumps** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Dumps` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

51. Hardware

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **hardware** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Hardware` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: `ST-OPS-012`; `ST-OPS-015`.

52. Sistema operativo

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sistema operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Sistema operativo` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: `ST-OPS-012`; `ST-OPS-015`.

53. Idioma

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **idioma** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Idioma` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015 .

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

54. Versión

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de **Versión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015 .

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

55. Branch

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **branch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Branch` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: `ST-OPS-012`; `ST-OPS-015`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

56. Manifest ID

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **manifest id** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Manifest ID` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: `ST-OPS-012`; `ST-OPS-015`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

57. Save afectado

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **save afectado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de `Save afectado` y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: `ST-0PS-012`; `ST-0PS-015`.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

58. Reproducción

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **reproducción** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.

6. Registrar en el ticket o release record el impacto específico de **Reproducción** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: **ST-OPS-012**; **ST-OPS-015**.

59. Registro de incidencias

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **registro de incidencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de **Player.log**, builds versionadas internamente y guardados JSON con primary, **.bak**, **.tmp** y **.recovery**. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.
6. Registrar en el ticket o release record el impacto específico de **Registro de incidencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: **ST-OPS-012**; **ST-OPS-015**.

Campo	Obligatorio
Incident ID	sí

Campo	Obligatorio
Fecha/hora y zona	sí
Fuente/canal	sí
Severidad/priority	sí
Build/manifest/branch	sí
Reproducción	sí para bug
Save/log/dump hash	cuando aplique
Owner	sí
Decisión y comunicación	sí
Release que corrige	al cerrar

60. SLA internos

Este capítulo forma parte del bloque **evidencia y registro** y define cómo se gobierna **sla internos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La base actual dispone de `Player.log`, builds versionadas internamente y guardados JSON con primary, `.bak`, `.tmp` y `.recovery`. Sin embargo, no hay recopilación automática de dumps ni portal de subida; cualquier evidencia debe solicitarse de forma explícita y respetuosa.

Regla operativa. Solo se solicita la información necesaria para reproducir y reparar. No se pide contraseña, correo de Steam, datos bancarios, nombre real ni archivos ajenos. Los saves se tratan como datos potencialmente sensibles y se conservan con acceso limitado.

Procedimiento mínimo.

1. Proporcionar instrucciones exactas para localizar log y save.
2. Redactar datos personales antes de adjuntar evidencia.
3. Asignar ID al incidente y al artefacto recibido.
4. Calcular hash y preservar copia inmutable.
5. Eliminar o anonimizar según la política de retención.

6. Registrar en el ticket o release record el impacto específico de **SLA internos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Paquete de evidencia con consentimiento, hash, versión, manifest, log, pasos y periodo de retención.

Fuentes Steamworks relacionadas: ST-OPS-012; ST-OPS-015.

Clase	Acuse objetivo	Triage objetivo	Decisión objetivo
S0	tan pronto como sea detectado	inmediato	retirada/contención en la misma sesión operativa
S1	mismo día laborable	mismo día	siguiente ventana razonable tras reproducción
S2	1-2 días laborables	2 días	incluir o rechazar con motivo
S3/S4	sin garantía pública	revisión periódica	backlog/closed

61. Tiempo de primera respuesta

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de primera respuesta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Tiempo de primera respuesta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

62. Tiempo de triage

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de triage** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Tiempo de triage** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

63. Tiempo de decisión

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **tiempo de decisión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Tiempo de decisión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

64. Comunicación inicial

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **comunicación inicial** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.

6. Registrar en el ticket o release record el impacto específico de **Comunicación inicial** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

65. Actualizaciones de estado

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **actualizaciones de estado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Actualizaciones de estado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

66. Resolución

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **resolución** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Resolución** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

67. Cierre

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **cierre** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe equipo 24/7. El plan debe ser viable para una producción individual y evitar compromisos que no puedan cumplirse. Los SLA son objetivos internos de respuesta y decisión, no garantías contractuales públicas.

Regla operativa. La primera comunicación reconoce el problema, delimita lo conocido y explica el siguiente punto de actualización. Nunca se promete una fecha de corrección sin reproducción, solución candidata y QA suficiente.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Cierre** y su relación con versión, build, save y comunicación.

Evidencia mínima. Cronología del incidente, mensajes publicados, tiempos internos y criterio de cierre.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

68. Hotfix

Este capítulo forma parte del bloque **SLA y comunicación de incidente** y define cómo se gobierna **hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Un hotfix corrige un defecto urgente con alcance mínimo. No incorpora features ni refactors no esenciales.

Regla operativa. Requiere reproducción, test de regresión dirigido, save/load, instalación/actualización y rollback. Incrementa PATCH cuando es público.

Procedimiento mínimo.

1. Confirmar recepción y severidad provisional.
2. Comunicar workaround solo si es seguro.
3. Fijar el siguiente update por condición o ventana realista.
4. Separar investigación, solución, validación y publicación.
5. Cerrar con versión corregida, notas y evidencia.
6. Registrar en el ticket o release record el impacto específico de **Hotfix** y su relación con versión, build, save y comunicación.

Evidencia mínima. Branch hotfix, diff, pruebas, manifest, notas y autorización.

Fuentes Steamworks relacionadas: ST-OPS-002; ST-OPS-003; ST-OPS-010.

69. Patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Patch` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

70. Update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Update` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

71. Content update

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **content update** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.

6. Registrar en el ticket o release record el impacto específico de `Content update` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

72. Save migration

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save migration** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El sistema actual solo acepta `CurrentSchemaVersion = 2`; no existe todavía una cadena de migradores históricos para una audiencia pública.

Regla operativa. Antes de cambiar schema se implementa migración versionada, idempotente, testeada desde cada versión soportada y con copia previa.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Save migration` y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de schemas, migradores, fixtures reales, rollback/fallback y estadísticas de resultado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

```
load original -> validate source schema -> create immutable backup
-> migrate one version at a time -> validate invariants
```

```
-> write new temporary file -> reload and compare  
-> commit atomically -> preserve recovery evidence
```

73. Backward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **backward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Backward compatibility` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

74. Forward compatibility

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **forward compatibility** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Forward compatibility` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

75. Versionado

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **versionado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.

3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Versionado` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

76. Branches

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **branches** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Branches` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

77. Release candidate

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **release candidate** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Release candidate` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

78. Previous stable

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **previous stable** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. La rama conserva la última build pública demostrada, no simplemente la penúltima subida.

Regla operativa. No se sobrescribe ni se elimina hasta que la nueva versión supere la ventana de observación y compatibilidad.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Previous stable` y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest, checksum, fecha, notas, saves de referencia y owner.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

79. Rollback

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam permite asignar builds anteriores, pero el rollback de binarios puede ser inseguro si el save ya migró.

Regla operativa. La decisión considera código, contenido, configuración y datos. Si no existe downgrade seguro, se prefiere hotfix forward o herramienta de recuperación.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.

6. Registrar en el ticket o release record el impacto específico de `Rollback` y su relación con versión, build, save y comunicación.

Evidencia mínima. Manifest anterior, compatibilidad de save, prueba de rollback y comunicación.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

80. Kill switch o feature flag

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **kill switch o feature flag** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Kill switch o feature flag` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

81. Build reproducible

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **build reproducible** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Build reproducible` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

82. Checksums

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **checksums** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Checksums` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

83. QA mínima de hotfix

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa mínima de hotfix** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `QA mínima de hotfix` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

84. QA completa de patch

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **qa completa de patch** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `QA completa de patch` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

85. Golden Path

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **golden path** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a MAJOR.MINOR.PATCH.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Golden Path` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

86. Campaña de siete días

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **campaña de siete días** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Campaña de siete días` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

87. Save/load

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **save/load** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.

3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Save/load` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

88. Instalación limpia

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **instalación limpia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Instalación limpia` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

89. Actualización desde versión anterior

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **actualización desde versión anterior** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Actualización desde versión anterior` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

90. Rollback de versión

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de versión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Rollback de versión` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

91. Rollback de datos

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **rollback de datos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0–15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Rollback de datos` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

92. SteamPipe

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **steampipe** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `SteamPipe` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

93. Depots

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **depots** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y BLD-016-PRE en PASS; BLD-016-POST, BLD-017-QA y BLD-H6 pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Depots` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-005; ST-OPS-006; ST-OPS-007.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

94. Manifest promotion

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **manifest promotion** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Manifest promotion` y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

La identificación técnica se conserva como tupla `app version + commit + Steam build + depot manifest + branch`, evitando que dos artefactos distintos compartan una etiqueta ambigua.

95. Notas de parche

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **notas de parche** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Steam ofrece un tipo específico de Patch Notes; cada build visible debe explicar magnitud y cambios relevantes.

Regla operativa. No ocultar cambios de save, known issues o riesgos. No inflar un hotfix como gran actualización.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.
5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de `Notas de parche` y su relación con versión, build, save y comunicación.

Evidencia mínima. Notas ES/EN vinculadas a versión, manifest y ticket.

Fuentes Steamworks relacionadas: `ST-OPS-004`; `ST-OPS-005`; `ST-OPS-006`; `ST-OPS-007`.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

96. Idiomas de las notas

Este capítulo forma parte del bloque **ingeniería de releases y parches** y define cómo se gobierna **idiomas de las notas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El repositorio conserva 20 registros de build: Sprints 0-15 y `BLD-016-PRE` en PASS; `BLD-016-POST`, `BLD-017-QA` y `BLD-H6` pendientes. SteamPipe, AppID, depots, branches y manifests públicos aún no existen.

Regla operativa. Todo cambio postlanzamiento se prueba primero en branch no default. Se conserva `previous_stable`, el manifest anterior y una copia de los datos necesarios. El hotfix limita alcance; un patch o update más amplio ejecuta regresión proporcional y campaña de save migration.

Procedimiento mínimo.

1. Crear rama y build desde commit limpio.
2. Incrementar versión conforme a `MAJOR.MINOR.PATCH`.
3. Generar checksum, manifest y release record.
4. Ejecutar reproducción, vecinos, save/load e instalación/actualización.

5. Promover manualmente tras Go/No-Go y conservar rollback.
6. Registrar en el ticket o release record el impacto específico de **Idiomas de las notas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Commit, versión, checksum, branch, manifest ID, pruebas, release notes, aprobación y rollback probado.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-005**; **ST-0PS-006**; **ST-0PS-007**.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

97. Comunicación de riesgos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunicación de riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Comunicación de riesgos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: **ST-0PS-001**; **ST-0PS-002**; **ST-0PS-003**; **ST-0PS-009**; **ST-0PS-010**.

98. Known issues

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **known issues** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Known issues** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

99. Comunidad

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comunidad** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Comunidad** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

100. Moderación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **moderación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Valve revisa contenido reportado, pero el proyecto debe vigilar el Hub y puede usar moderadores con permisos.

Regla operativa. Moderación por conducta y reglas publicadas. No borrar crítica legítima ni usar flags para desacuerdo.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.

6. Registrar en el ticket o release record el impacto específico de **Moderación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Log de moderación, regla aplicada, actor, fecha y posible apelación.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

101. Código de conducta

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **código de conducta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Código de conducta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

102. Spam

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spam** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Spam** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

103. Abuso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **abuso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Abuso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

104. Contenido ilegal

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **contenido ilegal** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.

4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Contenido ilegal** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

105. Spoilers

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **spoilers** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Spoilers** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

106. Solicitudes de reembolso

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **solicitudes de reembolso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El estudio no decide reembolsos individuales de Steam. Puede consultar informes agregados y motivos en los reportes de Steamworks.

Regla operativa. Responder con información de soporte, no presionar al jugador ni prometer resultado del reembolso. Usar tendencias para corregir expectativas o fallos.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Solicitudes de reembolso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Informe agregado, motivos categorizados y acción de producto.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

107. Comentarios negativos

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **comentarios negativos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Comentarios negativos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

108. Reseñas negativas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas negativas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Una review negativa puede reflejar bugs, valor, marketing o experiencia comunitaria. Steam recomienda no responder a todas ni discutir opiniones.

Regla operativa. Responder solo cuando exista información útil, una corrección publicada o un malentendido factual importante.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.

5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Reseñas negativas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Decisión de respuesta y texto breve revisado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

109. Reseñas positivas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **reseñas positivas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Reseñas positivas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

La acción pública debe ser proporcional, consistente y explicable; se conserva un registro interno sin publicar datos personales del usuario.

110. Respuestas públicas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **respuestas públicas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Respuestas públicas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

111. Transparencia

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **transparencia** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Transparencia** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

112. No prometer fechas sin aprobación

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **no prometer fechas sin aprobación** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.

3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **No prometer fechas sin aprobación** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

113. Roadmap público

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap público** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Roadmap público** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

114. Roadmap interno

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **roadmap interno** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Roadmap interno** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

115. Sugerecias

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **sugerencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Sugerencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

116. Votaciones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **votaciones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.

5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Votaciones** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

117. Alcance futuro

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **alcance futuro** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Alcance futuro** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

118. DLC

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **dlc** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **DLC** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

119. Expansiones

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **expansiones** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Expansiones** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

120. Demo

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **demo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.

5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Demo** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

121. Playtest

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **playtest** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Playtest** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

122. Achievements

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **achievements** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Achievements** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

123. Cloud

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **cloud** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Cloud** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

124. Localización futura

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **localización futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.

6. Registrar en el ticket o release record el impacto específico de **Localización futura** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

ES y EN se publican juntas para información crítica; una traducción pendiente no se sustituye silenciosamente por texto técnico o raw IDs.

125. Nuevas plataformas

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **nuevas plataformas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Nuevas plataformas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

126. Métricas de lanzamiento

Este capítulo forma parte del bloque **comunicación, comunidad y alcance futuro** y define cómo se gobierna **métricas de lanzamiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe Community Hub operativo ni roadmap público. Los sistemas futuros —DLC, expansión, achievements, Cloud, Playtest, nuevas plataformas— permanecen **NOT OPEN** o no comprometidos.

Regla operativa. La comunidad se modera por conducta, no por opinión. No se elimina crítica legítima, no se discute con reviews y no se anuncian fechas o features sin aprobación. El roadmap público solo contiene compromisos autorizados y suficientemente maduros.

Procedimiento mínimo.

1. Crear foros mínimos y reglas visibles.
2. Separar bugs, sugerencias y soporte.
3. Responder públicamente solo cuando aporte solución o aclaración factual.
4. Escalar abuso mediante herramientas de Steam.
5. Revisar cada anuncio contra la build y el alcance aprobado.
6. Registrar en el ticket o release record el impacto específico de **Métricas de lanzamiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Código de conducta, permisos de moderación, historial de acciones, claim matrix y copy aprobado.

Fuentes Steamworks relacionadas: ST-OPS-001; ST-OPS-002; ST-OPS-003; ST-OPS-009; ST-OPS-010.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

127. Crash-free sessions

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **crash-free sessions** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de `Crash-free sessions` y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: `ST-OPS-003`; `ST-OPS-013`.

128. Retención

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **retención** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.

5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Retención** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

129. Tiempo de juego

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de juego** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Tiempo de juego** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

130. Abandono

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **abandono** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Abandono** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: [ST-OPS-003](#); [ST-OPS-013](#).

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

131. Progreso

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **progreso** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Progreso** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

132. Economía

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **economía** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.

4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Economía** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

133. Stockouts

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **stockouts** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Stockouts** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

134. Softlocks

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **softlocks** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Softlocks** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: [ST-OPS-003](#); [ST-OPS-013](#).

135. Rendimiento operativo

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **rendimiento operativo** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Rendimiento operativo** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

136. Soporte

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.

3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

137. Volumen de tickets

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **volumen de tickets** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Volumen de tickets** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

138. Tiempo de respuesta

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **tiempo de respuesta** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Tiempo de respuesta** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: [ST-OPS-003](#); [ST-OPS-013](#).

139. Sentimiento

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **sentimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Sentimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

140. Reviews

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **reviews** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.

4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Reviews** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

141. Wishlists convertidas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **wishlists convertidas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Wishlists convertidas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

142. Privacidad de métricas

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **privacidad de métricas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Privacidad de métricas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

Se documenta denominador, ventana, zona horaria, origen y limitaciones; una cifra aislada no se interpreta como causalidad.

143. Telemetría opcional

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **telemetría opcional** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe instrumentación. Añadirla implica SDK, flujo de datos, proveedor, consentimiento o base aplicable, seguridad y mantenimiento.

Regla operativa. La telemetría es **NOT OPEN** hasta completar **26_Privacy_Data_and_Telemetry_Plan.md** y una decisión formal.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Telemetría opcional** y su relación con versión, build, save y comunicación.

Evidencia mínima. DPI/data map, contrato, política, opt-out, prueba y aprobación.

Fuentes Steamworks relacionadas: **ST-OPS-003**; **ST-OPS-013**.

144. No recolectar datos sin política

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **no recolectar datos sin política** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.

6. Registrar en el ticket o release record el impacto específico de **No recolectar datos sin política** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003; ST-OPS-013**.

145. Informes diarios

Este capítulo forma parte del bloque **métricas, privacidad e informes** y define cómo se gobierna **informes diarios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. No existe telemetría ni analytics en el proyecto actual. Las métricas inicialmente disponibles provendrían de Steamworks, soporte manual, QA y registros de build. Cualquier instrumentación propia requiere un plan de privacidad separado.

Regla operativa. No se recolecta un dato porque sea técnicamente posible. Cada métrica necesita propósito, definición, fuente, periodo, acceso, retención y decisión asociada. Las métricas agregadas no sustituyen investigación cualitativa ni justifican vigilancia innecesaria.

Procedimiento mínimo.

1. Definir diccionario de métricas.
2. Priorizar datos de plataforma y tickets antes de añadir SDK.
3. No incluir identificadores personales en informes ordinarios.
4. Revisar sesgos y muestras pequeñas.
5. Cerrar el ciclo: observación, hipótesis, cambio, QA y resultado.
6. Registrar en el ticket o release record el impacto específico de **Informes diarios** y su relación con versión, build, save y comunicación.

Evidencia mínima. Data inventory, base/política aplicable, dashboard reproducible, decisión y registro de acceso.

Fuentes Steamworks relacionadas: **ST-OPS-003; ST-OPS-013**.

146. Informes semanales

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **informes semanales** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Informes semanales** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

147. Retrospectiva

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **retrospectiva** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Retrospectiva** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-OPS-004**; **ST-OPS-006**; **ST-OPS-011**.

148. Mantenimiento

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **mantenimiento** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.

5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Mantenimiento** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

149. Dependencias

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **dependencias** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Dependencias** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

150. Unity y paquetes

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **unity y paquetes** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Unity y paquetes** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-OPS-004**; **ST-OPS-006**; **ST-OPS-011**.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

151. Vulnerabilidades

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **vulnerabilidades** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Vulnerabilidades** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-OPS-004**; **ST-OPS-006**; **ST-OPS-011**.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

152. Actualizaciones del motor

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **actualizaciones del motor** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.

4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Actualizaciones del motor** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

Los upgrades se realizan en rama aislada, con lockfile, changelog, licencia/NOTICE, build y regresión antes de decidir adopción.

153. Compatibilidad futura

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **compatibilidad futura** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Compatibilidad futura** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

El diagnóstico compara hardware, OS, drivers, resolución, branch y build; se evita atribuir el problema a la plataforma sin reproducción controlada.

154. Pruebas de regresión

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **pruebas de regresión** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Pruebas de regresión** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-OPS-004**; **ST-OPS-006**; **ST-OPS-011**.

155. Deuda técnica

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **deuda técnica** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Deuda técnica** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-OPS-004; ST-OPS-006; ST-OPS-011.

156. End of life

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **end of life** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El fin de vida no es abandono repentino; es una transición gobernada. Para un juego offline, la última build debe seguir siendo utilizable sin servicios innecesarios.

Regla operativa. Definir condiciones, periodo de aviso, soporte de saves, retirada de integraciones y archivo.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.

5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **End of life** y su relación con versión, build, save y comunicación.

Evidencia mínima. Plan EOL, aviso, build final, saves, notices y archivo.

Fuentes Steamworks relacionadas: **ST-0PS-004; ST-0PS-006; ST-0PS-011.**

157. Fin de soporte

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **fin de soporte** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Se diferencia de deslistado y de fin de venta. Puede cesar soporte activo manteniendo descarga y juego offline.

Regla operativa. No anunciarlo hasta decidir fechas, canales, excepciones de seguridad y destino de datos/tickets.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Fin de soporte** y su relación con versión, build, save y comunicación.

Evidencia mínima. Aviso multilingüe, calendario, FAQ y archivo.

Fuentes Steamworks relacionadas: **ST-0PS-004; ST-0PS-006; ST-0PS-011.**

158. Aviso a jugadores

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **aviso a jugadores** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Aviso a jugadores** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: ST-0PS-004; ST-0PS-006; ST-0PS-011.

159. Conservación de saves

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **conservación de saves** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Conservación de saves** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

Las pruebas usan fixtures reales anonimizados, primary/backup y estados límite; nunca trabajan sobre la única copia del jugador.

160. Disponibilidad offline

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **disponibilidad offline** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. El diseño actual es local/offline y no integra autenticación o servidores. Esa propiedad debe preservarse salvo decisión explícita.

Regla operativa. No introducir dependencia de red para iniciar, cargar, guardar o jugar el contenido base.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.

6. Registrar en el ticket o release record el impacto específico de **Disponibilidad offline** y su relación con versión, build, save y comunicación.

Evidencia mínima. Prueba sin conexión desde instalación limpia y tras reinicio.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

161. Archivado

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **archivado** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

1. Mantener inventario de dependencias y notices.
2. Probar cambios en rama aislada.
3. Conservar instaladores/manifests y saves de referencia.
4. Definir ventana de mantenimiento y criterios de EOL.
5. Archivar código, builds, documentación y decisiones.
6. Registrar en el ticket o release record el impacto específico de **Archivado** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

162. Work packages

Este capítulo forma parte del bloque **mantenimiento y fin de vida** y define cómo se gobierna **work packages** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Unity 6.3 LTS, URP 17.3.0 y los paquetes actuales están fijados, pero no existe aún un calendario postlanzamiento de actualizaciones, vulnerabilidades o fin de soporte. El juego es offline y debe conservar utilidad sin servicios online.

Regla operativa. No se actualiza motor o dependencia directamente en la rama pública. Se evalúan seguridad, compatibilidad, coste de migración, saves y rollback. El fin de soporte se comunica con antelación y no debe inutilizar guardados u operación offline sin necesidad.

Procedimiento mínimo.

- 1. Mantener inventario de dependencias y notices.
- 2. Probar cambios en rama aislada.
- 3. Conservar instaladores/manifests y saves de referencia.
- 4. Definir ventana de mantenimiento y criterios de EOL.
- 5. Archivar código, builds, documentación y decisiones.
- 6. Registrar en el ticket o release record el impacto específico de **Work packages** y su relación con versión, build, save y comunicación.

Evidencia mínima. Matriz de dependencias, evaluación de cambio, regresión, aviso de EOL y archivo verificable.

Fuentes Steamworks relacionadas: **ST-0PS-004**; **ST-0PS-006**; **ST-0PS-011**.

ID	Paquete	Gate
OPS-WP-01	Canales y soporte público	Pre-Coming Soon
OPS-WP-02	Incident register y plantillas	H6+
OPS-WP-03	Save migration framework	Pre-release
OPS-WP-04	Steam branches y rollback drill	Release candidate
OPS-WP-05	Patch/hotfix pipeline	Release candidate
OPS-WP-06	Community rules/moderation	Coming Soon
OPS-WP-07	Patch notes ES/EN	Pre-release

ID	Paquete	Gate
OPS-WP-08	Crash evidence strategy	Pre-release
OPS-WP-09	Cloud feasibility and conflict tests	Optional feature gate
OPS-WP-10	Metrics/privacy decision	Post-privacy-plan
OPS-WP-11	Maintenance/dependency cadence	Launch
OPS-WP-12	EOL/archive plan	Before end of support

163. Gates

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **gates** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Gates** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: [ST-OPS-015](#).

164. Checklists

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **checklists** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Checklists** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: **ST-OPS-015**.

165. Plantillas

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **plantillas** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Plantillas** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: **ST-OPS-015**.

166. Riesgos

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **riesgos** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Los riesgos principales son save corruption, rollback incompatible, actualización defectuosa, soporte desbordado, moderación reactiva, promesas públicas, Cloud prematuro, privacidad, paquetes y pérdida de trazabilidad.

Regla operativa. Cada riesgo tiene probabilidad, impacto, trigger, owner, mitigación, contingencia y evidencia de cierre.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Riesgos** y su relación con versión, build, save y comunicación.

Evidencia mínima. Registro de riesgos actualizado después de cada release o incidente.

Fuentes Steamworks relacionadas: **ST-OPS-015**.

167. Glosario

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **glosario** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Glosario** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: **ST-OPS-015**.

168. Historial de cambios

Este capítulo forma parte del bloque **ejecución y gobernanza final** y define cómo se gobierna **historial de cambios** después de una publicación. La regla se aplica tanto a incidencias reales como a ensayos internos, para que el primer lanzamiento no sea la primera vez que se ejecuta el procedimiento.

Estado actual. Este plan es preparatorio. Su valor depende de convertir capítulos en work packages, gates, checklists, plantillas y revisiones vivas. Ningún elemento postlanzamiento está operativo por el mero hecho de estar documentado.

Regla operativa. Cada gate necesita owner, evidencia y decisión manual. Las plantillas reducen errores, pero no reemplazan criterio. Los riesgos se revisan tras cada incidente, parche, cambio de Steamworks o modificación sustancial del producto.

Procedimiento mínimo.

1. Asignar work packages y dependencias.
2. Definir gates de Ready/Done.
3. Ensayar checklists en una release interna.
4. Versionar plantillas y glosario.
5. Actualizar historial con motivo, evidencia y rollback.
6. Registrar en el ticket o release record el impacto específico de **Historial de cambios** y su relación con versión, build, save y comunicación.

Evidencia mínima. Backlog, matriz de gates, paquete de plantillas y changelog aprobado.

Fuentes Steamworks relacionadas: **ST-0PS-015**.

Anexo A. Fuentes oficiales de Steamworks verificadas

Las siguientes páginas se verificaron el **1 de julio de 2026**. Se conservan como fuentes operativas, pero deben revalidarse en Steamworks antes de activar una feature o cambiar un proceso.

ID	Fuente	URL	Uso
ST-0PS-001	Steam Community	https://partner.steamgames.com/doc/features/community	Hub availability, developer identity, forums and moderation responsibilities.
ST-0PS-002	Community Moderation	https://partner.steamgames.com/doc/marketing/community_moderation	Flagging, official responses, moderator practice and escalation to Steamworks Support.
ST-0PS-003	User Reviews	https://partner.steamgames.com/doc/store/reviews	Review rules, developer responses, review-bomb escalation and feedback interpretation.
ST-0PS-004	Updating Game Build	https://partner.steamgames.com/doc/sdk/updating	Upload update to a test branch before moving it to default.
ST-0PS-005	Branches (Betas)	https://partner.steamgames.com/doc/store/application/branches	Password branches, client opt-in, default branch behavior and test workflow.
ST-0PS-006	Builds and Manifests	https://partner.steamgames.com/doc/store/application/builds	Build/depot/manifest model, default authorization and beta branches.
ST-0PS-007	Uploading to Steam	https://partner.steamgames.com/doc/sdk/uploading	SteamPipe ContentBuilder, builds, depots and rollback to previous uploads.
ST-0PS-008	Steam Cloud	https://partner.steamgames.com/doc/features/cloud	Auto-Cloud/API, per-user paths, quotas, developer-only testing, dynamic sync and cloud logs.

ID	Fuente	URL	Uso
ST-0PS-009	Small Update / Patch Notes	https://partner.steamgames.com/doc/marketing/event_tools/type_patchnotes	Patch-note event type and communication expectations for every pushed build.
ST-0PS-010	Events and Announcements	https://partner.steamgames.com/doc/marketing/event_tools	Library/store/community communication for patches and updates.
ST-0PS-011	Updating Your Game — Best Practices	https://partner.steamgames.com/doc/store/updates	Cadence, announcements and major-update visibility guidance.
ST-0PS-012	Steam Error Reporting	https://partner.steamgames.com/doc/features/error_reporting	Legacy crash reporting is nearing end-of-life and limited to Windows 32-bit.
ST-0PS-013	Developer Refund Reporting	https://partner.steamgames.com/doc/finance/refunds	Refund volumes and player-provided reasons in Steam financial reports.
ST-0PS-014	Managing Steamworks Users	https://partner.steamgames.com/doc/gettingstarted/managing_users	Users, groups, permissions and moderator access.
ST-0PS-015	Contacting Steamworks Support	https://partner.steamgames.com/doc/help/contact	Official support escalation route for partner issues.

Anexo B. Evidencia de implementación observada

Evidencia	Estado
Producto/versión	Cartridge & Cloud / 0.0.17
Empresa configurada	VRM Games
Resolución configurada	1024×768, resizable 0
Guardado integrado	JSON, primary + backup + temp + recovery, generación y validación
Schema	IntegratedGameStateSnapshot.CurrentSchemaVersion = 2
Steamworks	no encontrado en código/paquetes
Steam Cloud	no encontrado
Telemetría/analytics	no encontrado
Crash SDK externo	no encontrado
HTTP/network client de producto	no encontrado
Paquetes directos	41

Escenas de build observadas

Enabled	Escena	GUID
sí	Assets/_Project/Scenes/Bootstrap.unity	ada1e52ac6e09b9468e2109ccc7cbfc0
sí	Assets/_Project/Scenes/MainMenu.unity	b00786c6952a55d418cf16503030c766
sí	Assets/_Project/Scenes/Store.unity	45adcce906944f74bad3a0fb7d62f8e5
sí	Assets/_Project/Scenes/TestLab.unity	002b2ad62af3c614b91e32722c9452be

Archivos técnicos hashados

Archivo	Byte s	SHA-256
Assets/_Project/Scripts/Infrastructure/Persistence/JsonIntegratedSaveRepository.cs	13424	8a5408551acedf29a900d04ba627f82345f7f99e2d0c6ad439f0cff0b7a88106
Assets/_Project/Scripts/Infrastructure/Persistence/IntegratedSaveJsonCodec.cs	31736	d38c0c5b67e14d3f4fd81813d5f22d31759e45ac70f73ed5eff8a22593143656
Assets/_Project/Scripts/Infrastructure/UIUX/AtomicJsonFile.cs	2065	ead9646924fe5361a035cfacbb0be8ab92beff9d510e8d393b175286ba49020a
Assets/_Project/Scripts/Application/Persistence/IntegratedSaveContracts.cs	3093	2f70dacb1588aade0236f7286fffa268fa438ad0b36977a9c6164f173db84b10
Assets/_Project/Scripts/Domain/Persistence/IntegratedGameStateSnapshot.cs	20654	af49722b2ffdac7370ffa90bf104430e830b9ed2c69ac4952b2929478a547f62
Assets/_Project/Scripts/Infrastructure/GameSession/JsonSaveGameRepository.cs	11042	4841b006e79fece72618b5d9569b875307bbd8143c0003f86d74935e5e6d1e39
Assets/_Project/Settings/BuildProfiles/Windows_Development.asset	1946	63e658288030c33844a84c37fc636fa908e9d8fbe44d2180aefcf03a0e5c93fa
Assets/_Project/Scenes/StoreInitial.unity	90851	ca9689130ae8ad29a04af7145864451213a81d60c249c761ab10bd2401e4ea57
Assets/_Project/Scenes/Store.unity	88611	f8ceebcb55d1a97ca0a1205363d31f8641107e52968b9ab18aefe72d01e43a90
Assets/_Project/Scenes/TestLab.unity	29023	b390fbb375d5f9fb97563afb01fc3e6c0e612b4463411fb6060ac63293c4332a

Anexo C. Inventario de documentos consolidados

Documento	Bytes	SHA-256
00_Enfoque_y_Alcance.md	127401	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f
01_Game_Design_Document.md	132822	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177
02_Vertical_Slice_Specification.md	64531	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
02_Vertical_Slice_Specification.preview.md	64531	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f
03_Technical_Design_Document.draft.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
03_Technical_Design_Document.md	101994	66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
04_Modelo_de_Datos.draft.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
04_Modelo_de_Datos.md	102948	d851a72b55742c2c704cc7c002929bc5894e7142511c6af843440bb193fb19d4
05_UX_Flow.md	56805	e08da5ff67fb073a8b950babe325ae58ddb2e121513dc2f553acd4f2c14b867e
06_Production_Roadmap_y_Sprint_Plan.md	61564	d00b156c0ad1c66069278119c55f76c738a577a3e2835f770208d1a5f2faadff
07_QA_Testing_Plan.md	79435	bc9c05a3737e345546ef16e5390e034295ef35ad66c31647852aec81b4e7affe
08_QA_Testing_Matrix.xlsx	185032	233e57f01d2468fe13c136c8fcd3ee75b39bd3de6530349626ea98d4ff254214
09_CSharp_Coding_Standards.md	95321	3b734d6cd49f31c09c10bb4941625e143f6c634e3fb50c157f0720e73b1c2a68
10_Unity_Project_Setup_Guide.md	78408	31430bb0544edd9577323aa0009d6ca3df8b9fbcf6c8a150fc33df69b9f963a4
11_Build_y_Versioning_Guide.md	85324	38a16a0f97505c0b405509dd3c7e3d1b50e77b89fbe3f4cb1931d3e04f602215
12_Excel_Maestro_de_Produccion.xlsx	158100	405376cb64f49b34f1f842f3840c12654e9f08e2ec4534b5c149fa811e87dd83
13_Trazabilidad_y_Control_de_Cambios.xlsx	189424	f674e21d1a3a1f0970a3d339f26f2a2b51a19b5c257c24b9d3fa4118a5cf50eb
14_Project_Binder_Indice_Maestro.md	92828	0c24dcd47e0d44793e75e764e3fc2c146e5b1fea04d887d43feb0df558711559
15_Guia_Maestra.md	119920	879bc1925cccf77e10df6c104302f3afc72f71e8c58dd85ae4f06e84e6f8b624
16_Auditoria_Global_de_Coherencia.md	87439	4a62376baee623be40957ae0511d2dcab2919947f3abb2cef3502b9bb60863e
17_Art_Bible.md	133913	095c6cc42223a972c7faa68435595fc7c22f839fab62b2038371a0ef6c14239
18_Audio_Bible.md	144525	c2eabe93a7c26f70e1d4d77fb0ddcaa97e2cdd1e70247e637b9db5bd2f07ce92
19_UI_Style_Guide.md	149800	b34ca4db1eba7a536466b6a9a105d8d92396ade2108aa7dd99fa7d5d4eaebc1a
20_Economy_and_Balance_Specification.md	154547	908c70772ed35eecd05c3036c6901523a4caf2c3e708ca8f049c2a160ef2aac
21_Initial_Content_Catalog.xlsx	174383	9047299f60cfbd368fc4a2c8bee8d0ff4132dc5642363a9a67d1523559eb9ed3

Documento	Bytes	SHA-256
22_Localization_Plan.md	157323	184a53b2813bc1327a6913e6e88ea4c81142da1ee4ddef8f75467c784f1ff0
23_Legal_Credits_and_Licenses_Register.xlsx	465135	f543b3b5de8900f405910fac5d72697045f8c4969d98fce4c1cf4f12e532b2f1
24_Steam_Publishing_Plan.md	171037	f087e24d3a18f8c4a6651728d5f70d1df804042587e206fdeac2e7e637a87646

Anexo D. Inventario histórico y operativo preservado

Se inventarían **241** fuentes relacionadas con release, QA, builds, soporte, incidentes, persistencia, Steam y gobernanza. No se omiten las versiones anteriores: sus hashes permiten distinguir reediciones idénticas y cambios reales.

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.3	PDF	Documentation/00_Official_Baseline/v0.3/01_Design/Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.pdf	102712	9539e4a8e097a84c413a8b65a8bc77d2d2a859eeab9c9697e84473ce67025ab7
0.3	PDF	Documentation/00_Official_Baseline/v0.3/01_Design/Cartridge_And_Cloud_GDD_v0.4_PCSteam.pdf	292627	9d07c1cbfde60aa02403129f5b0b2b36852cfb514123e836d8c8e81ee7fa837d
0.3	PDF	Documentation/00_Official_Baseline/v0.3/01_Design/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.1.pdf	37172	86b0a49dac832eda01fa751b5a9b720610868ba389093d2a8784222d01326ac3
0.3	PDF	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.3.pdf	25562	81e38d9861f5cae0d4448c0aff214e0ffc922e82e2491f8b37cf737719bd97a
0.3	PDF	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_Modelo_de_Datos_v0.3.pdf	201297	7909e47fb733c5129b9d5ead2743b9d82e75bb4a592f931cfaa61a419a671380
0.3	PDF	Documentation/00_Official_Baseline/v0.3/02_Technical/Cartridge_And_Cloud_TDD_v0.3_Technical_Design_Document.pdf	59929	de5a87992375dbef16f3f4ea2013262c31f08b184d074e8b952b8a124a6b8afb
0.3	PDF	Documentation/00_Official_Baseline/v0.3/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.3.pdf	28949	2b48b54738598dbaa9b461d7cad6f4a6a2f70bb8589df55e72424ec7c9e40e6e
0.3	XLS X	Documentation/00_Official_Baseline/v0.3/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Matrix_v0.3.xlsx	21821	f7d48e93b2ca3cc8e86b1dfc2164126d923af38ba0c76cb4dde566e6bc4da05d
0.3	PDF	Documentation/00_Official_Baseline/v0.3/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.3.pdf	39300	2d71d4dd720598429744bc6cf959162a8915301ad69171bc6816168e47f75709
0.3	PDF	Documentation/00_Official_Baseline/v0.3/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.pdf	22534	e358622a237b57cdcd65fb380460d70eafa0ef1085b43bf5e8fb7dff3ee734f0
0.3	PDF	Documentation/00_Official_Baseline/v0.3/05_Business_Release/Cartridge_And_Cloud_Localization_Plan_v0.3.pdf	27103	461add7f8bf216439351ce4afe63b573de50e8bae3f9e270992917f616d82ad8
0.3	PDF	Documentation/00_Official_Baseline/v0.3/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.pdf	23140	10ff2852e8621c210bb578f63181382665ccbedeba3d62bfdce7eb4cb4e53472
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.3.md	3678	fcc0988f428b45303c4cc2f5a681cbcd9970475667bb6a37fa97a36529dc3705
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.md	30444	a79f65cb5d960ff1e8fa6c83b8369b295547600f2f058163078f07a3b892c9e4
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_GDD_v0.4_PCSteam.md	114534	8939cf748467b18cee2e351b5eac309fe108b7fe02cd6da59a96230e9dd125fe

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.md	3972	ffc5bc6af65b27ede021acfddef2dbc5f20f680560e0be1a872586ab146a2f101
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Localization_Plan_v0.3.md	3806	f48dcd0d33e717b3e65734a9f87677d3b72edb9e67b54ff630720ecc46686349
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Modelo_de_Datos_v0.3.md	71576	6d3fd4346cf55ecd33d5aae3e1c1f267936e594d8902d3c51f3edb76003a46b
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.3.md	5738	93b345037c1356c7e176a7377bd66a168aa356366aa3c4e16062b09fcb46ee60
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.3.md	9293	517a90d50868dd91010be4eaaca335326b63e8d370613c82a5b5664753ab3891
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.md	3923	1a004be75991bdcbbaf457a57da123e3297fce15a38c591e3f589e11e277dbd5
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_TDD_v0.3_Technical_Design_Document.md	17342	86160690276d137118b119d6376eee8d1834097dbfe92fb6da7d18356d737eb2
0.3	MD	Documentation/00_Official_Baseline/v0.3/99_Source_Markdown/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.1.md	8682	69792ba4790022b23d631d437e043d77844f775b94d6552bc33bc682b876bb0
0.4	PDF	Documentation/00_Official_Baseline/v0.4/01_Design/Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.pdf	90214	75b1684eaf50b546febcb2608230524385fb5f7f685be8d885c0cfff309389af643
0.4	PDF	Documentation/00_Official_Baseline/v0.4/01_Design/Cartridge_And_Cloud_GDD_v0.4_PCSteam.pdf	233208	daf3d2376783c00533ce932e3da58ddddd77b65fab1efa2aa7a141241dc125069
0.4	PDF	Documentation/00_Official_Baseline/v0.4/01_Design/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.2.pdf	53623	d2505a3ba89c696248ddad9d0e3c4c7516eb0cf93b124111f2a4e3c6291bc46e
0.4	PDF	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.4.pdf	46424	b26821c7646a0648c9bca240688bc7e3f82edd0c5ff853ac115e88ee3a742c4c
0.4	PDF	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_Modelo_de_Datos_v0.3.pdf	167248	7b23bfd9335d0cca818896512b434579a5c94b20eac7185a6928e02e19175ab9
0.4	PDF	Documentation/00_Official_Baseline/v0.4/02_Technical/Cartridge_And_Cloud_TDD_v0.4_Technical_Design_Document.pdf	75576	b0f63b8bfc72cb8b5a2671c6bdd3b51f8d79fffd681ae77be7539245f320ca
0.4	PDF	Documentation/00_Official_Baseline/v0.4/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.4.pdf	45968	a633ddefdface673e33411b00fd52a84d6984fecf5a49586352d2efb95ef4951
0.4	XLS X	Documentation/00_Official_Baseline/v0.4/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Matrix_v0.4.xlsx	26049	80d79fa25df8c4daf0e91ff3f0e519ffcd283d802cf9e418e06fa88fa01d4d4
0.4	PDF	Documentation/00_Official_Baseline/v0.4/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.4.pdf	59968	047a7bdd0a0b1ed6f9dec50aacc429bf1f18a2ba874fca0a28a270cc61bbfa45
0.4	PDF	Documentation/00_Official_Baseline/v0.4/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.pdf	42670	e4105813b235d24b75edf539eae505342f58f1f26a4b087e870dcbda9d63f558
0.4	PDF	Documentation/00_Official_Baseline/v0.4/05_Business_Release/Cartridge_And_Cloud_Localization_Plan_v0.3.pdf	43688	fc6cb711af7107fecd82d8963f6461a6d2f00941e08d93ea535c16c8ce917c17
0.4	PDF	Documentation/00_Official_Baseline/v0.4/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.pdf	43916	b052a99f72f3b531b75b5950e5459fc6bd2a62c516be7dc1ad5e53d3a4684ec3
0.4	MD	Documentation/00_Official_Baseline/v0.4/06_Development_Records/Governance/Project_Foundation_Release_Record.md	466	f789933c54973d329e078dd5a561b43f174333653a36aca00ce5254daaac0aab
0.4	MD	Documentation/00_Official_Baseline/v0.4/06_Development_Records/Sprint_00/S0.10_Windows_Final_Build_Record.md	1484	2bddd6ad5bfff23d564e9d179c0b31b5baa4dcf00624539571485725734231a2
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.4.md	4723	6ac0c7072b5ac4b1844d82daf7e1ec5f241dba369145e5c62152e290b8309acd
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Economy_Balance_Specification_v0.1.md	30549	f83578295bab346c04186abd7d5ca64c00656561f738e71b8c229605b717fabcb

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_GDD_v0.4_PCSteam.md	114639	88be82c3d3cfa4cb5965379e621b0aba522efa1217041cdc23a129ec4817485a
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.3.md	4077	1a0e921f139b73179fe734eec452dcc3aa8c43860309390fe41ea1fb588fe81f
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Localization_Plan_v0.3.md	3911	2158be2c46ec697e617419a9192bbaa8a0dd4435b5c650fca925921b958cb3a
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Modelo_de_Datos_v0.3.md	71681	3c722995fa9423f0de31cf9a45c5f527574c5f77eb13f2788db0b84585baf8a0b
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.4.md	6571	e9976daa630e8bc1460831eea0c3d280d24f38d3907ef3dd97baec6e35661404
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.4.md	10116	623c83bd390d3fb1e7cb09e554f6a29f63db662c7ca3e296c65938c65969a724
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.3.md	4028	56979c2830951e4a82a7d01793cebe9a930df4a77325de2b7489d0e73cb0777
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_TDD_v0.4_Technical_Design_Document.md	19073	bd2237fa0323821c4d4717d89940f005bbce2c2232ac91a4dd72f54301fb879d
0.4	MD	Documentation/00_Official_Baseline/v0.4/99_Source_Markdown/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.2.md	9317	884464e9599f0f1b66fdef7a3c6422bf9d37b883817f480e0429ebdcd126b9d
0.5	PDF	Documentation/00_Official_Baseline/v0.5/01_Design/Cartridge_And_Cloud_Economy_Balance_Specification_v0.2.pdf	19491	57a71ee88a5a25d9afe1a94d798a0d6522e1007c6dfb1d91aa98089a7cbcf c3
0.5	PDF	Documentation/00_Official_Baseline/v0.5/01_Design/Cartridge_And_Cloud_GDD_v0.5_PCSteam.pdf	23195	7061f5ee8528d2fdffc2c219ff8a54f997705fe38cf3a713856caf519a997264
0.5	PDF	Documentation/00_Official_Baseline/v0.5/01_Design/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.3.pdf	18610	b256eb82b1c49fe2598797ec5ad74d4e4a22523ebba62ef77ca27a777d231fc
0.5	PDF	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.5.pdf	18588	34fc32adf9c6383d0bf5b79ed446cd4dd00c9518cbd67c6e9d7cd5623b236347
0.5	PDF	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_Modelo_de_Datos_v0.4.pdf	23279	022bbbed24274749a627b0547d51d9e6570dcf711194c747394a3979241c6314a
0.5	PDF	Documentation/00_Official_Baseline/v0.5/02_Technical/Cartridge_And_Cloud_TDD_v0.5_Technical_Design_Document.pdf	23003	87ca947c5d4fc56096449d54e59f4b3abea4daa028ce41a00cb3162d72284d24
0.5	PDF	Documentation/00_Official_Baseline/v0.5/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.5.pdf	21719	d75c34e0085a41c5a875fd17cf8fe3f7ea3f6b6ba6e496246a2cf93e06427b93
0.5	XLS X	Documentation/00_Official_Baseline/v0.5/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Matrix_v0.5.xlsx	8327	dbff9b69bb8edae19c2158fb4293aaf574a584a407cca672cf6f89dabae6690e
0.5	PDF	Documentation/00_Official_Baseline/v0.5/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.5.pdf	20203	0763d0d411746199eaa762acc995f591393eb39a1c8d88511c4dec488e56e74e
0.5	PDF	Documentation/00_Official_Baseline/v0.5/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.4.pdf	15779	d8fa9f63a454facc7e8d4e3fa8e50d1a95f0bb7a8f8bef06211e9d12accb5343
0.5	PDF	Documentation/00_Official_Baseline/v0.5/05_Business_Release/Cartridge_And_Cloud_Localization_Plan_v0.4.pdf	15507	0194a14af0cb3121540d5ac73f3264c2d29037256953c15a74379690e1727e4c
0.5	PDF	Documentation/00_Official_Baseline/v0.5/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.4.pdf	16309	f0382da1359aa1339758e067dc58c2d921c46ca37aef870c74ecce0ef320a3b2
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_01/QA/SI_Acceptance_Matrix.md	1455	a5a4e0a192fe33559c582efd9ee4d53170e26af9e07db63aae1e22b1f5c66f49
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_01/QA/SI_QA_Execution_Record.md	1065	f9ae3d8cb7bd112af8d518e93ccfe03fbce2309d555f13130b0325427ef68daad

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/Documentation/S2_2.4_PostImplementation_Record.md	1370	6d8abe4b6e4fda5c39ffd60d97011f543f82a7a104941401d40120785709fc4b
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/Governance/ADR-0011_Minimal_Versioned_Save_Skeleton.md	2505	4048b071ec41b0d5caf44beb0d5a4ff7547e409f566a403d0e9331a604de076dd
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/QA/S2_Acceptance_Matrix.md	993	1c184fd5c44de6051e83614676a1571d4d49e7cd8b4d68c8ef85167b64e2692f
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_02/QA/S2_QA_Execution_Record.md	1827	70ad9d5ab6b32071b69fcb35e7f76889b95b4d84ec6c6e6174ab2d793ccea4f
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/Documentation/S3_3.4_PostImplementation_Record.md	1620	cf1d302783bfb7156655cc269fad60d1457fbac83048709a2d2e5b3591111969
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3.4_Build_Execution_Record.md	1303	bdbcbe899c627a7abf1f302f7db42ce29de1f0809c8373a454e3bc070984ab7d
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3_Acceptance_Matrix.md	1643	dcd3a753ec03bf462ed61fee96551716970f05a54702fcec5ebdc271bda15b7
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_03/QA/S3_QA_Execution_Record.md	2573	c36caf9cccc1d762ab9f201d7eae4f2e07ae88f1787d7ed0df6bf13305850116
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/Documentation/S4_4.4_PostImplementation_Record.md	1186	561dc5549c7c89740905004bd43c03d73737f0c5b8c63843784712cebcf07828
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4.4_Build_Execution_Record.md	1164	6f662c5fe7cce7a3080584798ccf51c720d4ade1dc50b1be2f8c4c107da529d
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4_Acceptance_Matrix.md	2209	30a86d51980351f806ef30aa6187ce90857bde474c163d47a6a3396b60522f89
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_04/QA/S4_QA_Execution_Record.md	2537	0c5ec51188fd94404436c7093e01ed0a0d0d75fc466da104e777fd7494aa0603
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/Documentation/S5_5.4_PostImplementation_Record.md	1625	80578149d99d8d19960522a7ad446f1da1464c98f5d7ab300f2e8083ba6f3a30
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5.4_Build_Execution_Record.md	1347	83700192228af99fb4487bdf4cdd1332d13a194be7cd5fb197278013952f8fe1
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5_Acceptance_Matrix.md	2498	3c265e9509b133370788c1ea024e4de481f773b526a524f2ec880d60abe053b6
0.5	MD	Documentation/00_Official_Baseline/v0.5/06_Development_Records/Sprints/Sprint_05/QA/S5_QA_Execution_Record.md	2607	28e52cfc1189efc63cafabf899b0b94924bf2fb3edb92a43973797fcb77616d4
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.5.md	2384	3df0fc5321a57c528f6386bcd922c2e930ba194736af94f064262c371bb63be6
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Economy_Balance_Specification_v0.2.md	2773	06c11a0e1ea6a93a2f3fd18794007ce2160fbbec1658d43456d4b16fecdc8c484
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_GDD_v0.5_PCSteam.md	4208	17284e1e8d67332d5a8e76e56d194b7e176225dfb49e2729caf6664a96c6aa09
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.4.md	2001	18f78b69a56edee6f8783d32859d6e231d061422d24abd6feefdbf06fe68567b
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Localization_Plan_v0.4.md	1892	01a3ddb286f975644985bc0c034eab75ef2ba4a283cd200f2e5dfd673732839f
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Modelo_de_Datos_v0.4.md	3193	e657c304a8f9572abfd1e8240c23860b4feb42b9b314f6384d31993f2f5a01a3
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.5.md	3654	d0623a03b2ff75fa95ecd6bbaa5fcbab0fa8faada31fc2a6b4a569caa50f0e285
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.5.md	2853	c19ab67f26aae4b44fd5c6d431fb163ddbfccfcbcc0ca68518826040b534f1c

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.4.md	2025	19664d4092ea89898c5bc37483a0b1aba6f473abbfa02c2794df8f2dd482058d
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_TDD_v0.5_Technical_Design_Document.md	4246	2067d16cac0aae694c69895371dca234c4daaa5258f13922844214283bd09e25
0.5	MD	Documentation/00_Official_Baseline/v0.5/99_Source_Markdown/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.3.md	2813	17213417be3b7b2b95f8e3841d09a6679eee689bb62a35335e3f318135c15e46
0.5	MD	Documentation/00_Official_Baseline/v0.5/CURRENT_PROJECT_HANDOFF.md	2275	9c251d0f3c14584f5da35a50ab3a5b62564e6f1f656d903fa2590eda74e2e396
0.6	PDF	Documentation/00_Official_Baseline/v0.6/01_Design/Cartridge_And_Cloud_Economy_Balance_Specification_v0.3.pdf	37357	814b045831b60bd22488cbb9b255a87e9c714abb9f0c9716227004af6334b38b
0.6	PDF	Documentation/00_Official_Baseline/v0.6/01_Design/Cartridge_And_Cloud_GDD_v0.6_PCSteam.pdf	43453	f7002d60df83e5ed854264b4a802324de145e6389d3ec1de3b9f02ca4f619acd
0.6	PDF	Documentation/00_Official_Baseline/v0.6/01_Design/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.4.pdf	38854	326720d1a0d1302142034421279586f319992a9ecd3772c1922b6f5be085a692
0.6	PDF	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_Build_Versioning_Guide_v0.6.pdf	37901	758372f65f3e817c8cc192d557cac69f2ecb2b22d0705e8feeac9e5034cf0a02
0.6	PDF	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_Modelo_de_Datos_v0.5.pdf	38808	296fbc062ddb5cb16dad4f5c98bccb1301c437eaeac7b58a23d51009d8f32fe3
0.6	PDF	Documentation/00_Official_Baseline/v0.6/02_Technical/Cartridge_And_Cloud_TDD_v0.6_Technical_Design_Document.pdf	44591	47cb69c540b382355f767b5cef1cbecc0f4ed29cf5711be34cefe196007b50d9
0.6	PDF	Documentation/00_Official_Baseline/v0.6/04_QA_Production/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.6.pdf	37116	5e1a0fea9e45f7af252162baba7544778134216c0f9c839aabbdd48687b3c370
0.6	XLS X	Documentation/00_Official_Baseline/v0.6/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Matrix_v0.6.xlsx	8110	96a6eb8272a2c9a42f718f71406a1b8d770e2d7cf0d96d2c3d0a0c418445d681
0.6	PDF	Documentation/00_Official_Baseline/v0.6/04_QA_Production/Cartridge_And_Cloud_QA_Testing_Plan_v0.6.pdf	38292	ff451a919545374e6bd22a44efd7f8e65d3e9001741a2ddcac58b4f339052c04
0.6	PDF	Documentation/00_Official_Baseline/v0.6/05_Business_Release/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.5.pdf	28754	eee199263d089442dc5ea6228b10b562d7930163c707a7e90402e734bf94db51
0.6	PDF	Documentation/00_Official_Baseline/v0.6/05_Business_Release/Cartridge_And_Cloud_Localization_Plan_v0.5.pdf	28785	0a7e40cfb0a39f46a303f93fd89ea65073e507955c15d89a67639ae1f445aa8
0.6	PDF	Documentation/00_Official_Baseline/v0.6/05_Business_Release/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.5.pdf	28734	a21ea9abe67f1d825731d25b8608c8f7b67e5e1b437356e0e1ee3d97870d7728
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	1241	7a297d84a36ecd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Handoff/NEXT_CHAT_START_HERE.md	896	e0cca56b839dc2071363b310c06543178dbf978622990fc7f4be301e077fc754
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Historical/Sprint_16_Integration_Timeline.md	1783	89fde821c460328fbb8700022883006d2eb8a1ab892c266972b7a1cc207cbc0
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/Documentation/StoreInitial_Authoring_Plan.md	741	f67d195bcd2bd1f861dcf5c53d0a299260c1f04c2f299250856a76616131eba
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/Governance/Sprint_16_Current_Status.md	665	bc336c8bf852d942d1b0289cebffdde9db889088990fcc6151df5db559e94d1a9
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/Governance/Sprint_16_Representative_Asset_Integration_Record.md	554	91133cf2823d732d58fcd23601d6816e10e00375cc38e39737929d88fc4ec54
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/QA/S16_Acceptance_Matrix.md	572	4ad529aceeaf7e2ea87def0e0be2f8f3998d6218f6ac11a06b40f770e66c44fd
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/QA/S16_QA_Execution_Record.md	328	40c6de3168db5dcf5c71afc166c0a3a2bf34f2dc3e71344c3ccea6176d573766

Baseline/área	Tipo	Fuente	Bytes	SHA-256
0.6	CSV	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_16/Traceability/S16_Traceability_Entries.csv	460	3263ee5adec14e81588139e9c291ae973115ac75bef792336a514ac3643fd67
0.6	MD	Documentation/00_Official_Baseline/v0.6/06_Development_Records/Sprints/Sprint_17/Governance/Sprint_17_Opening_Brief.md	239	fb2c1a557e3b6cd765497315fa554eacc048c2fd21a2be9c7f5f28895a189ac1
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Build_Versioning_Guide_v0.6.md	2462	8cd0f62e233cfbad2bed3a07122696e2ea81b4fd5280987cffb9a8a2a1e98d43
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Economy_Balance_Specification_v0.3.md	2564	0f3a938d2dcfb83e03b20cc8b7902342d8124f42965cb9585c81a8854ce16abc
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_GDD_v0.6_PCSteam.md	4005	333c7777584bcdde29c1caa8f90f4e43e3013e88cc339466a1a51f5bdd42cd7
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Legal_Credits_Licenses_Register_v0.5.md	1951	610fc2ca1dd8ea1859fb779039e5de841610646d1e5601d316b2fe34410c3c81
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Localization_Plan_v0.5.md	1891	e8c63ec8a67f5d87f5dcec078c94ac08620b27a064a5a6866a20fa78de1becf9
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Modelo_de_Datos_v0.5.md	3168	409201b990309744b05a65a3306034e605e7254d10a5d4232b707cdcbc8779b4a
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Production_Roadmap_Sprint_Plan_v0.6.md	3191	f7b50c87cbc0faa2478f677f868e2e29dd1e74abfe74c1cbb30aed74ab7ac70a
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_QA_Testing_Plan_v0.6.md	2637	7659a777a9aa78f083d8a6d577898bec89430cae108b079ee7137c769da7f069
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Steam_Publishing_Plan_v0.5.md	1866	f7840ee791e07430dd750500421dc6e262d21d5119e634634919247ff13cf8d0
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_TDD_v0.6_Technical_Design_Document.md	4268	daa5578e670e61ffe40db754ef512d02885d1bb7600c74487194ad8ec27cb2ce
0.6	MD	Documentation/00_Official_Baseline/v0.6/99_Source_Markdown/Cartridge_And_Cloud_Vertical_Slice_Specification_v0.4.md	3027	efff0d3056515ad3e71a9c751f7ec86ce932d3098fb5870b246c04559695a3f5
0.6	MD	Documentation/00_Official_Baseline/v0.6/CURRENT_PROJECT_HANDOFF.md	1241	7a297d84a36ecd9e4296a631fc5acbff882218d125e63312850e8f28b46cd87
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/00_Visual_Targets/ChatGPT Image 29 jun 2026, 21_37_54.png	1977934	cfdd490b742396ac2ff6fb6a0b1e6cea8d71fb2005305dd9c82487ec7ea88b84
working/current records	FILE	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/01_Architecture/.gitkeep	0	e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/02_Furniture/ChatGPT Image 29 jun 2026, 21_38_14 (10).png	1767735	42175ffa89a40d6bfb57217b38a5dfee04e337bc44b838ca8db01812e56454e5
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/02_Furniture/ChatGPT Image 29 jun 2026, 21_38_14 (7).png	1670316	6d5032c38dc50979430763f7ee18a84deee243325d3400affc2012551864ee1a
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/02_Furniture/ChatGPT Image 29 jun 2026, 21_38_14 (8).png	1996425	78f6c5d8ecc65077ee7f191715e71f31b15a7ce22b59becf54e69dcca19bded48
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/02_Furniture/ChatGPT Image 29 jun 2026, 21_38_14 (9).png	1575644	16f940c84ce6c14a277a5ed236d1306df17d80419e118507f0f2032f0b4ad558
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (1).png	1590353	410d0bf38ab8848df5a496e250786d793d1d45bf8bbeab48c8f8ca83fa91d8fd
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (2).png	1786999	6107ce0119514adab534d271d224c0a3b9b2006eebe551cc43297299a9edc404
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (3).png	1954907	27639fb594bb9c24fc20403cbc1f5a683b91af6b12250d3062fb2d0ec054528c
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (4).png	1939499	7ade04828ed3b7623c7e7dc271adde637cdcbf746c6a78566a3faa3cb5073e63

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (5).png	1901472	48be40e282d818092991e3f24d9e6b21e324025374a15ae762faf4fca8762779
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/03_Products/ChatGPT Image 29 jun 2026, 21_38_14 (6).png	1774013	56891223f619f891a523c01037823ca04fa386cc2b0963858c6115e9984df4bd
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (1).png	1728315	fd439e31144c4f1eedd8d417911c1f78fbdbe02623b4859535db50082e8a491
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (2).png	1685372	e5bca43f10a16a56d793260cf1bad77fa308db981207d5e6d37325159e6d8a11
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (3).png	1651401	093b91e470d56a2609622789d9955d99f700014a6e9733fb4dc32da839d6d7e2
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (4).png	1521051	546539231dbaf0ac0f0d3bd92bd4349e7ea7c13bee7dd1aa0768363529198c07
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (5).png	1854890	1d02083cf9c0736568b4fbefea98d57e1104d84e1def585b1c8bf87fec8462d1
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (6).png	1675117	6f0416232df549d46156a2f6ff92d039b785cf7aa44f82f17c840d05bc1927a6
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (7).png	1684465	ebb549522b8d6ac4ca57d47c8bf1d8cd0e96608824dfb5f0f2d7bd51b6c2dd98
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/04_Characters/ChatGPT Image 29 jun 2026, 21_38_50 (8).png	1902077	3f1d37c6c690d4173bc0067a6460e29bc5bce2e8911c965d6a7b8aae05d296cf
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (1).png	2230307	ce23b3b1e2bf6186ab627de7247b794ea9ed6f8b80bd84cb02df94619801f09
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (2).png	2385231	79e9ccc8ba8126f7a63b5f38fdc4bafcbab5479b39e6882de1add15f66529e7f
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (3).png	2195489	78cb05222f8155d5bd9e709e8e214f4517023444ed79538c3693bb0b09ea232f
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (4).png	2065962	1074b34d1051c2e32fa23325f042158d04e7497b54d61c44f4cc1c3b4505b1f5
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (5).png	2198732	686585e4a4b1054b23cdea08297e28ddd0df211d966dc0ab8a6116ce9c17ca25
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (6).png	2232046	7c349ed7b9258e832c0e7c3f3529fb4fe8ea322fd9d7c52ef004d98a741b21519
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (7).png	2038776	cf9a87944e4aaff1730c4162d300a6204d248f4e6a62f8c0fe46ee7152819ff5
working/current records	PNG	Documentation/01_Working_Documents/Sprint_16/Art_3D/Concept_Art/05_Expansion_Concepts/ChatGPT Image 29 jun 2026, 21_39_17 (8).png	1962162	cb6d28e87a016aa78cb5b6d1d84dbe42368fad7b50f3ef8345214bf1e6a59dd
working/current records	MD	Documentation/10_Development_Records/Builds/S0.10_Windows_Final_Build_Record.md	1484	2bddd6ad5bff23d564e9d179c0b31b5baa4dcf00624539571485725734231a2
working/current records	MD	Documentation/10_Development_Records/Handoff/CURRENT_PROJECT_HANDOFF.md	1168	ea48666ee43dbf978bef364ee645cd3ff32f31e4e6fdb96b9edfab316546028
working/current records	MD	Documentation/10_Development_Records/Sprint_01/QA/S1_Acceptance_Matrix.md	1455	a5a4e0a192fe33595c582efd9ee4d53170e26af9e07db63aae1e22b1f5c66f49
working/current records	MD	Documentation/10_Development_Records/Sprint_01/QA/S1_QA_Execution_Record.md	1065	f9ae3d8cb7bd112af8d518e93ccfe03fbce2309d555f3130b0325427ef68daad
working/current records	MD	Documentation/10_Development_Records/Sprint_02/Documentation/S2_2.4_PostImplementation_Record.md	1370	6d8abe4b6e4fda5c39ffd60d97011f543f82a7a104941401d40120785709fcd4b
working/current records	MD	Documentation/10_Development_Records/Sprint_02/Documentation/S2_2.4_PostImplementation_Template.md	460	b03fed839b37fa856b30c1b9255248bbe9363ac314b43fe8a737bcee151c1c01

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	MD	Documentation/10_Development_Records/Sprint_02/Governance/ADR-0011_Minimal_Versioned_Save_Skeleton.md	2505	4048b071ec41b0d5caf4beb0d5a4ff7547e409f566a403d0e9331a604de076dd
working/current records	MD	Documentation/10_Development_Records/Sprint_02/QA/S2_Acceptance_Matrix.md	993	1c184fd5c44de6051e83614676a1571d4d49e7cd8b4d68c8ef85167b64e2692f
working/current records	MD	Documentation/10_Development_Records/Sprint_02/QA/S2_QA_Execution_Record.md	1827	70ad9d5ab6b32071b69fcb35e7f76889b95b4d84ec6c6e174ab2d793ccea4f
working/current records	MD	Documentation/10_Development_Records/Sprint_03/Documentation/S3_3.4_PostImplementation_Record.md	1620	cf1d302783bf7156655cc269fad60d1457fbac83048709a2d2e5b3591111969
working/current records	MD	Documentation/10_Development_Records/Sprint_03/QA/S3.4_Build_Execution_Record.md	1303	bdbcbe899c627a7abf1f302f7db42ce29de1f0809c8373a454e3bc070984ab7d
working/current records	MD	Documentation/10_Development_Records/Sprint_03/QA/S3_Acceptance_Matrix.md	1643	dcd3a753ec03bf462ed61fee96f551716970f05a54702fcec5ebdc271bda15b7
working/current records	MD	Documentation/10_Development_Records/Sprint_03/QA/S3_QA_Execution_Record.md	2573	c36caf9cccc1d762ab9f201d7eaeef2e07ae88f1787d7ed0df6bf13305850116
working/current records	MD	Documentation/10_Development_Records/Sprint_04/Documentation/S4_4.4_PostImplementation_Record.md	1186	561dc5549c7c89740905004bd43c03d73737f0c5b8c63843784712cebcf07828
working/current records	MD	Documentation/10_Development_Records/Sprint_04/QA/S4.4_Build_Execution_Record.md	1164	6f662c5fe7cce7a3080584798ccf51c720d4ade1dc50b1be2f8c4c107da529d
working/current records	MD	Documentation/10_Development_Records/Sprint_04/QA/S4_Acceptance_Matrix.md	2209	30a86d51980351f806ef30aa6187ce90857bde474c163d47a6a3396b60522f89
working/current records	MD	Documentation/10_Development_Records/Sprint_04/QA/S4_QA_Execution_Record.md	2537	0c5ec51188fd94404436c7093e01ed0a0d0d75fc466da104e777fd7494aa0603
working/current records	MD	Documentation/10_Development_Records/Sprint_05/Documentation/S5_5.4_PostImplementation_Record.md	1625	80578149d99d8d19960522a7ad446f1da1464c98f5d7ab300f2e8083ba6f3a30
working/current records	MD	Documentation/10_Development_Records/Sprint_05/QA/S5.4_Build_Execution_Record.md	1347	83700192228af99fb4487bdf4cdd1332d13a194be7cd5fb197278013952f8fe1
working/current records	MD	Documentation/10_Development_Records/Sprint_05/QA/S5_Acceptance_Matrix.md	2498	3c265e9509b133370788c1ea024e4de481f773b526a524f2ec880d60abe053b6
working/current records	MD	Documentation/10_Development_Records/Sprint_05/QA/S5_QA_Execution_Record.md	2607	28e52cfc1189efc63cafabf899b0b94924bf2fb3edb92a43973797fcb77616d4
working/current records	MD	Documentation/10_Development_Records/Sprint_06/Documentation/S6_PostImplementation_Record.md	1055	8e05e7d148c68c5af0f700283b34d533081bc3e7ce63dea745ea2169640093f1
working/current records	MD	Documentation/10_Development_Records/Sprint_06/Governance/ADR-0026_Sprint_06_Version_And_Persistence_Boundary.md	587	88e809b7e5a461f57c5d9f943adaf6eeb0445f62ba3cd08440f5d2e45c48ed1f
working/current records	MD	Documentation/10_Development_Records/Sprint_06/QA/S6_Acceptance_Matrix.md	3014	b55fe039c6404ff22749ff07f20dc741ee277ece3a892d9d1a6a526eb61dab8e
working/current records	MD	Documentation/10_Development_Records/Sprint_06/QA/S6_QA_Execution_Record.md	2097	a7a14df3c2dc18a98c1354176d48502bafbd3309a61ea8bff3846db4c348e7c
working/current records	MD	Documentation/10_Development_Records/Sprint_07/Documentation/S7_PostImplementation_Record.md	1407	92d67e8e9150e7f53042e3bc42d69828b73c2dde23f6730e9758e8bd6a69731
working/current records	MD	Documentation/10_Development_Records/Sprint_07/QA/S7_Acceptance_Matrix.md	3351	e5517ab2f2b047aacb634c1ee7e42ea6f4b1b0f3bb6477cc7a0fb595439546b2
working/current records	MD	Documentation/10_Development_Records/Sprint_07/QA/S7_QA_Execution_Record.md	2132	ed5696f59956c5341e66f5d4bd7c9ff10b1c2ae56c47f0e70f7acd412d62626
working/current records	MD	Documentation/10_Development_Records/Sprint_08/Documentation/S8_PostImplementation_Record.md	2173	34f827046ab4402cb0aca95a90695727e6a4654052245511e2130416053da878
working/current records	MD	Documentation/10_Development_Records/Sprint_08/Governance/S8_Handoff_Update_Template.md	1297	752c345e77607a3d096548520f2ce082a816d2eeb0d01ea0d29ddb2bd32013a

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	MD	Documentation/10_Development_Records/Sprint_08/QA/S8_Acceptance_Matrix.md	1833	514b68f2839e76b1757df9d2d457dca9701cbe8f3de4169c6e38fe7e7c1e576c
working/current records	MD	Documentation/10_Development_Records/Sprint_08/QA/S8_Build_Execution_Record.md	1531	975e35974c3ac193dedc354ad39f652dccd31356a4d69ec241dddcfdd84f29c1
working/current records	MD	Documentation/10_Development_Records/Sprint_08/QA/S8_Compilation_Incident_001_NUnit_Assert_Multiple.md	1971	c3f9557dbe19d9326df55919c56712a4c7b57a77e3d1044708bce974705cb69c
working/current records	MD	Documentation/10_Development_Records/Sprint_08/QA/S8_QA_Execution_Record.md	1991	6b648c6839cb4cd0f249f4318624210896b63a393cd888a3b1ac2d9f70edc61b
working/current records	MD	Documentation/10_Development_Records/Sprint_09/Documentation/S9_PostImplementation_Record.md	798	6b07d5adf234fe07af6475b6abac1fed856e6b4238fa14ae4e5daf731e73123
working/current records	MD	Documentation/10_Development_Records/Sprint_09/Governance/S9_Handoff_Update_Template.md	501	e3d501b14918d549aab4d42cc2d289a8f1d83730600d1083109ea64ea409baae
working/current records	MD	Documentation/10_Development_Records/Sprint_09/QA/S9_Acceptance_Matrix.md	727	476e8342efea9323b73b46337890b795fe2b3bc500486f008a9e9f5dbf6706ef
working/current records	MD	Documentation/10_Development_Records/Sprint_09/QA/S9_Build_Execution_Record.md	773	743edb2e5421984273fb169aab3fbb16c48d6b0ac2e177c5e83ba01b51099304
working/current records	MD	Documentation/10_Development_Records/Sprint_09/QA/S9_QA_Execution_Record.md	661	ee2c2033cb369301abe3eb39bf1f0d3a71c241431b1889651e14a895437d114a
working/current records	MD	Documentation/10_Development_Records/Sprint_10/Documentation/S10_PostImplementation_Record.md	782	c3fa683ba9a345654be078d3ddb4e17e6dea32b20515674f8f991f97cd4424ee
working/current records	MD	Documentation/10_Development_Records/Sprint_10/Governance/S10_Handoff_Update_Template.md	429	e7893c392feb7b35d3e270034b36f9da758d73673adb7e3f50cecea5763003a0
working/current records	MD	Documentation/10_Development_Records/Sprint_10/QA/S10_Acceptance_Matrix.md	665	8a3558314d7d868e3e2ce0dc33fe62e7b1b33eddff1e70b2d6eab43c9b3600d8c
working/current records	MD	Documentation/10_Development_Records/Sprint_10/QA/S10_Build_Execution_Record.md	535	adea9b7fcdab6e319f0ded3171260cc0200e2ae3ab424916a8068a7175872369
working/current records	MD	Documentation/10_Development_Records/Sprint_10/QA/S10_QA_Execution_Record.md	544	c45fb3dd29d4ec4dc67ad42f496932c94cbe384096c560c08d3d1c9615c6a5fe
working/current records	MD	Documentation/10_Development_Records/Sprint_11/Documentation/S11_PostImplementation_Record.md	589	5642f0de8091f082726d0d25fda968e886c16221dee482cdc1cf047006b5abf1
working/current records	MD	Documentation/10_Development_Records/Sprint_11/Governance/S11_Handoff_Update_Template.md	422	3da63a8600ab4da7f1afe4032690763029d3551297fad0d51afbfa841687dad5
working/current records	MD	Documentation/10_Development_Records/Sprint_11/QA/S11_Acceptance_Matrix.md	650	a6f876fb02398be1ee466d2f9dde9613ad9c0fa2fc0088778b791ab9a3d00e8d
working/current records	MD	Documentation/10_Development_Records/Sprint_11/QA/S11_Build_Execution_Record.md	494	6e5b3ee3b6b9a1a6d2f6c90fc07df567cfd81126f6e2c06aa4c3307db206f24
working/current records	MD	Documentation/10_Development_Records/Sprint_11/QA/S11_QA_Execution_Record.md	457	e09346a52a005447f98244392575dce8bfe0e1c24395d87ef4efa994958222d
working/current records	MD	Documentation/10_Development_Records/Sprint_12/Documentation/S12_PostImplementation_Record.md	420	2e0156fac6fda2d0475d71f8f3c66f827bc8b5ca5325d8e99348522ae72040bb
working/current records	MD	Documentation/10_Development_Records/Sprint_12/Governance/S12_Handoff_Update_Template.md	361	683e99d27f7af84f319fad7c98230f6dec7f9b9eeddff8b4066f750b22c230bd0
working/current records	MD	Documentation/10_Development_Records/Sprint_12/QA/S12_Acceptance_Matrix.md	661	9b16fac6f71a8d05203d140a77a1b0c202a5f445556cc663208b04e5c45392b9
working/current records	MD	Documentation/10_Development_Records/Sprint_12/QA/S12_Build_Execution_Record.md	431	e3531d9bc28665f33ed38c176188246308d38bd480be42c1758f848e433f69b4
working/current records	MD	Documentation/10_Development_Records/Sprint_12/QA/S12_QA_Execution_Record.md	402	e53fa611bf41782ecb96b8d41010b2607f3e38223dad09898a92bdfec0f6ba9f

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	MD	Documentation/10_Development_Records/Sprint_13/Documentation/S13_PostImplementation_Record.md	624	0b5d8860dc3625f95cd33eb8e44f3aeb551d887271c034e77a35481ed40b1799
working/current records	MD	Documentation/10_Development_Records/Sprint_13/Governance/S13_Handoff_Update_Template.md	431	7d91ce81d3567cd4f392f6d9784c7415d628eec2b65e7468ea5e1b694af743f8
working/current records	MD	Documentation/10_Development_Records/Sprint_13/QA/S13_Acceptance_Matrix.md	693	e7f134005949b363e84fadbd28d740ff268adb3c42de47070fbbc968862319f2b
working/current records	MD	Documentation/10_Development_Records/Sprint_13/QA/S13_Build_Execution_Record.md	493	887232c7346268536f197fc6069ad7f518735c2f51c302f0d27e6c2625cd005b
working/current records	MD	Documentation/10_Development_Records/Sprint_13/QA/S13_QA_Execution_Record.md	456	a0ecdcbd34ae6a3e96f2ff4b19602fcfdad3628da0f4647a96631aecce9bd9e0
working/current records	MD	Documentation/10_Development_Records/Sprint_14/Architecture/ADR-0056-CompatibleIntegratedSaveV2.md	198	5cc539efb691df4398f23002613d08079fa8869bde84816f89a91be781ffcf6a
working/current records	MD	Documentation/10_Development_Records/Sprint_14/Documentation/S14_PostImplementation_Record.md	559	301104a8d7d45f8a80047b4e7d6f7f7fc5f633418341784bd08c744e41603066
working/current records	MD	Documentation/10_Development_Records/Sprint_14/Governance/S14_Handoff_Update_Template.md	437	70ac9acc5473a29f44723f4e1499f027f5adb772ae45635ebdb035f2374bd4f
working/current records	MD	Documentation/10_Development_Records/Sprint_14/QA/S14_Acceptance_Matrix.md	761	6abc7f9f6b1f199f797ede8e4610be0819c89c52ba763b09489f76e5b6d2b3e9
working/current records	MD	Documentation/10_Development_Records/Sprint_14/QA/S14_Build_Execution_Record.md	490	758b82a0301e5f3de699b76365cf74c577377bcb6d88333c4b65a26e6065af33
working/current records	MD	Documentation/10_Development_Records/Sprint_14/QA/S14_QA_Execution_Record.md	407	61083207815690667077272760527c8529f70916831a1bd7d75cc58125663a7f
working/current records	MD	Documentation/10_Development_Records/Sprint_15/Architecture/ADR-0063-ClosedDayAutosaveIdempotency.md	200	8575a38534155149713d4c711e65216ce43402c06dce60d181fbad87d806ad59
working/current records	MD	Documentation/10_Development_Records/Sprint_15/Documentation/S15_PostImplementation_Record.md	1276	7d942b11542da16a2fba0e01edd20a7fc4bd43000f1b48e49c23bc7b6f389cd0
working/current records	MD	Documentation/10_Development_Records/Sprint_15/QA/S15_Acceptance_Matrix.md	744	ef5c3d5a5d788d7a52965e858620ca76d260711f96e5132bb7938c765b6762da
working/current records	MD	Documentation/10_Development_Records/Sprint_15/QA/S15_Build_Execution_Record.md	359	cb74c7605f9918b31b4f49ebbe0f63004b03886d5468bd258a3eb6f83ea9b61a
working/current records	MD	Documentation/10_Development_Records/Sprint_15/QA/S15_QA_Execution_Record.md	373	317dd72193364df1dd104ef8be879fa22dbd4f8374e3e0bd454fef8f0881277d
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Governance/Sprint_16_Two_Phase_Charter.md	640	b191b8b5d47654141df7860d6a79430552cc68e1bcd786d1d4fef55583dc605
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0067-Sprint16TwoPhaseDelivery.md	214	59f759e966b057f091473d52cf205d1a089a0917d2f8fff80342ca8ce2ca0a75
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0068-PurePhase1Sidecar.md	232	5a4b767d6daf552a48dc2b4d256b38c698da130b41314243902c4b5fbab68459
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0069-AutosaveCoordinatedCheckpoint.md	250	c4aaad9ce98b604d52894350668f7e59cc8c9d893ec3e96658a46b7bf0dbd44a
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0070-PlacementCompatibilityBridge.md	353	66a5b5172267c6984edd5474ba03dc2fb0da28de72c310ced74feba060f6ddf17
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0071-DecoupledPresentationFallbacks.md	223	f2e025be2f68b6de2388bd63a6b25549d4e6d111012bc685b034fa120ff9616c
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0072-SharedMaterialAndPrefabIds.md	223	f32e2b4b274522d03df3c1c08084d86114a19dbe15fa2b3008c217943e2ff11a
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Architecture/ADR-0073-CompletedCheckoutSnapshotRecords.md	377	7d401e745e557c63167dae48cd0f4b05b1d08f5c58a2e652b4321925fc35ca7a

Baseline/área	Tipo	Fuente	Bytes	SHA-256
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Documentation/S16_P1_Implementation_Record.md	819	3538c025e899624c8dfb10c740bcb06387f5be41a78a3396a8f5937a5cfc147
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Documentation/S16_P1_PreImplementation_Record.md	496	0d6915434ed6282040553f42f849399d6a8e3380eb4e38a6082ea3d13fba0e9c
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Governance/S16_P1_Charter.md	874	4b5190df961a1180ce9fed6ced5fe194e88998f5f65df176bea9104c7bc825dc
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/Governance/S16_P1_Current_Status.md	569	6494de60296d252da3368dab5a5850938c4a5c219d414f80bde328379f1c7f3b
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_Acceptance_Matrix.md	1130	d6d2ca42287be0fced9c3d9da09df5db9729442157035a9a711e0ddb9d91337
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_QA_Execution_Record.md	443	a044ba4e7ebb4af622c74397e2a80ff6fe59ad7e9c1bf89ddb62f5b06bcacec7
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_Test_Plan.md	836	ff35438f414f45a152e04532664e348d0fed9bae380c9dc70bf520af5c7c04c8
working/current records	MD	Documentation/10_Development_Records/Sprint_16/Phase_1/QA/S16_P1_Validation_Checklist.md	646	ac85fc7fe9c1e905ca5ea2eb245aa016acf8c611b2a3fe1a10fb3d0740e0198b
working/current records	CSV	Documentation/10_Development_Records/Sprint_16/Phase_1/Traceability/S16_P1_Asset_Inventory.csv	6002	c91f4676f6c8b07c075fda4bf3563530380a5e35770d9f68b5497b77cf99641b
working/current records	CSV	Documentation/10_Development_Records/Sprint_16/Phase_1/Traceability/S16_P1_Traceability.csv	1635	d1e5ada71ed4d81b6a067aa05ea35626d5156967a1f8b212da829be32a4d318d

Anexo E. Matriz de preparación postlanzamiento

Dominio	Ready	Done	Estado actual
Soporte	canales, owner, privacidad, plantillas	simulacro completo y capacidad real	NOT READY
Incidentes	severidades, register, evidence	S0/S1 drill ejecutado	PARTIAL
Hotfix	branch, versioning, tests	hotfix interno publicado y rollback probado	NOT READY
Save migration	schemas y fixtures	migración desde todas las versiones soportadas	NOT READY
Steam branches	AppID, depots, permisos	qa/rc/previous_stable verificadas	NOT OPEN
Comunidad	Hub, reglas, moderadores	prueba de publicación/moderación	NOT OPEN
Métricas	diccionario y privacidad	informes reproducibles sin datos excesivos	NOT OPEN
Mantenimiento	dependencias y cadence	primer ciclo de patch cerrado	PLANNED
EOL	criterios y archivo	aviso y build final preservados	FUTURE

Fin del documento.

Fuente: 25_Post_Launch_and_Live_Operations_Plan.md · SHA-256: `158e1e43514c5d5d6c365f6cd1a73929f54c4a48c5cc460b593530ae5e49bac9`

Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.